

**A
SUMMER INTERNSHIP REPORT
On**

**Predicting Customer Response to
Telemarketing Campaigns**

Submitted

by

Rabadiya Gopeshkumar Harsukh

211430116017

Guided By:
Prof. Roshni Mandli

INFORMATION TECHNOLOGY DEPARTMENT

*A Summer Internship Submitted to
Gujarat Technological University In fulfillment for the award of degree
of Bachelor of Engineering in Information Technology*

ACADEMIC YEAR – 2024-2025



NEW L. J. INSTITUTE OF ENGINEERING AND TECHNOLOGY

Pakwan, Behind Rajpath Club Gate to Sindhu Bhavan Road,
Sarkhej - Gandhinagar Highway,
Ahmedabad, Gujarat 380054

CERTIFICATE



This is to certify that the Internship report submitted along with the project entitled Internship in **Su-kEm Tech Lab** has been Completed by **Rabadiya Gopeshkumar Harsukh** under my guidance in complete fulfilment for the Bachelor of Engineering in Information Technology Branch, 7th Semester of Gujarat Technological University, Ahmedabad during the academic year 2024-25

Date:

Place: NEW LJiet, Ahmedabad.

Signature and Name of Guide

Prof. Roshni Mandli

Assistant Professor, (IT),

IT Department,

NEW LJiet (143), Ahmedabad.

Signature and Name of H.O.D.

Dr. Gayatri Pandi

Associate Professor, (IT),

IT Department,

NEW LJiet (143), Ahmedabad.

Seal of Institute



DECLARATION

I hereby declare that the Internship report submitted along with the Internship entitled Internship in **Su-kEm Tech Lab** submitted in Complete for Bachelor of Engineering in **Information Technology** Branch to Gujarat Technological University, Ahmedabad, is a bonafide record of original Internship work Completed by me at **Su-kEm Tech Lab** under the supervision of External Guide **Vrunda Hingrajiya** and that no part of this report has been directly copied from any students' reports or taken from any other source, without providing due reference

Name of Student

Rabadiya Gopeshkumar Harsukh

Signature of Student

ACKNOWLEDGEMENT

I wish to express my sincere gratitude to my External guide **Vrunda Hingrajiya** for continuously guiding me at the company and answering all my doubts with patience. I would also like to thank **Dr. Gayatri Pandi** (H.O.D. of IT Department) for motivating me every time whenever I get confused, I would also like to thank my Internal Guide **Roshni Mandli** for helping me through my internship by giving me the necessary suggestions and advices along with their valuable co-ordination in completing this Internship.

I also thank my parents, friends and all the members of the family for their precious support and encouragement which they had provided in completion of my work. In addition to that, I would also like to mention the company personals who gave me the permission to use and experience the valuable resources required for the Internship.

Thus, in conclusion to the above said, I once again thank the staff members of **Su-kEm Tech Lab** for their valuable support in completion of the Internship.

Thank You

Predicting Customer Response to Telemarketing Campaigns

Enrollment No.: 211430116017

Student Name: Rabadiya Gopeshkumar Harsukh

NEW L. J. Institute of Engineering and Technology (143)

Semester: VII, Information Technology Department

Abstract

This project focuses on analyzing bank telemarketing data to extract insights and enhance marketing strategies. By examining customer demographics, call history, and campaign outcomes, the project aims to uncover patterns that influence the success of telemarketing campaigns. Additionally, the project involves developing predictive models to identify potential customers who are more likely to subscribe to banking products or services. The findings contribute to the optimization of telemarketing efforts, allowing banks to tailor their strategies for improved customer engagement and campaign effectiveness.

Key-Words

- 1) Project Background
- 2) Data Cleaning
- 3) Exploratory Data Analysis
- 4) Data Visualization
- 5) Machine Learning: Classification
- 6) Machine Learning: Regression
- 7) Conclusion & Recommendation

Modules

1. Project Background:

This section provides an overview of the project, including the problem statement, objectives, and the importance of the project. It sets the context for why the project is being undertaken and what you aim to achieve by the end of it.

2. Data Cleaning:

Data cleaning involves preprocessing the data to remove or correct any errors, inconsistencies, or missing values. This step is crucial to ensure the quality and reliability of the data, which directly impacts the accuracy of any analysis or models built later.

3. Exploratory Data Analysis (EDA):

EDA is the process of analysing datasets to summarize their main characteristics, often using visual methods. It helps in understanding the data, identifying patterns, spotting anomalies, and forming hypotheses that can guide further analysis.

4. Data Visualization:

Data visualization involves creating graphical representations of the data to make it easier to understand and interpret. It helps in identifying trends, patterns, and insights that might not be apparent from raw data. Common tools for data visualization include charts, graphs, and plots.

5. Machine Learning: Classification:

Classification is a type of supervised learning where the goal is to predict a categorical label for a given input based on training data. Examples include spam detection, sentiment analysis, and

image recognition. Algorithms used for classification include logistic regression, decision trees, random forests, and support vector machines.

6. Machine Learning: Regression:

Regression is another type of supervised learning used to predict a continuous output variable based on one or more input features. Examples include predicting house prices, stock prices, or any other numerical value. Common regression algorithms include linear regression, polynomial regression, and ridge regression.

7. Conclusion & Recommendations:

This section summarizes the findings of the project, discusses the implications of the results, and provides recommendations based on the analysis. It often includes suggestions for future work or how the insights gained can be applied in practice.

Each of these sections plays a critical role in a data science project, from understanding the problem and preparing the data to analysing it and drawing actionable insights.

TABLE OF CONTENTS

CHAPTERS	PAGE NO.
Title Page	i
Completion Certificate	ii
Declaration	iii
Acknowledgements	iv
Abstract	v
Table of Contents	viii
CHAPTER 1 INTRODUCTION	1
1.1 Introduction	1
1.2 Company Profile	1
1.3 Company Products	2
1.4 Company Mission and Vision	4
CHAPTER 2 Role and Responsibilities During Internship	5
2.1 Objective	5
2.2 Introduction to Python Programming	5
2.2 Role and Responsibilities During Internship	6
CHAPTER 3 Internship Work	8
3.1 Internship Work	8
CHAPTER 4 Conclusion	57
4.1 Conclusion	57
CHAPTER 5 Future Enhancement	58
5.1 Future Enhancement	58

CHAPTER: 1

INTRODUCTION

1.1 Introduction

Su-kEm Tech Lab specializes in creating high-quality products across various development fields, including Android, iOS, Windows, web development, enterprise solutions, and UI/UX design. They aim to enhance users' lives with their innovative solutions. The company, founded by CEO Vrunda Hingarajia, is based in Ahmedabad, India. For more information, visit [Su-kEm Tech Lab](<https://sukem.in>).

1.2 Company Profile

Su-kEm is a software development company focusing primarily on Mobile Application Development and Web Development, providing top-quality solutions according to client needs.

1.3 Company Details

Company name:

Su-kEm Tech Lab

Address:

A-40, Shaktidhara complex, Near kotak Mahindra bank, Near Dinesh Chambers, Tollnaka Bapunagar, Ahmedabad-380024.

Website URL:

<http://www.sukem.in/>

Company Type:

Software Development

HR name:

Vrunda Hingrajiya

HR Contact Number:

9638930696

HR Email:

info.sukem@gmail.com

Internal Guide/Developer Name:

Vrunda Hingrajiya

Designation Of Internal Guide/Developer:

Business Executive

Contact Number:

9638930696

1.4 Company Products

Developer Drive:

As mentioned above, this is a perfect option if you're just beginning your career in web development. By web developers, for web developers, Developer Drive offers the latest news in the world of web development. This includes opinion articles, tutorials, tips & tricks to start off on the right foot. Are you looking to learn how to minimize downtime? How about ensuring your site is secure and easy for consumers to navigate? Or would you like to keep tabs on every Microsoft software release? From JavaScript to mobile development, Developer Drive has you covered.

Simple Programmer:

While Simple Programmer was originally created by founder John Sonmez to distill what he learned as a web developer into understandable content, today it is focused on helping software developers, programmers and other IT professionals improve their careers and their lives. This blog gets into the nitty-gritty of being a web developer and if you've wondered about something web development related, there's a good chance they've written about it. Simple Programmer puts a fun spin on videos, keeping you engaged with humor while providing valuable content. They're constantly posting new articles – almost daily – so you'll never be bored.

CodeNewbie:

Just like the blog title says – if you're a code newbie looking for insight on all things web development, you're in the right place. This site acts as a virtual community for anyone new to the coding world. CodeNewbie offers an open look into the lives of web developers and serves as a place for people to read about career advice while hearing stories from people on their coding journey through the site's podcasts.

CodeNewbie originally started as a weekly TwitterChat to connect people learning to code. While they still host a weekly #CodeNewbie TwitterChat every Wednesday at 9PM EST, it's evolved into a supportive, international community of people learning to code.

Scotch.io:

So, you've been a web developer for a while, or maybe you're just a fast learner. Either way, hats off to you! Some of the 'beginner' blogs outlined above may be a bit too junior for you, which is where Scotch.io comes in handy. This blog is full of tutorials on everything from JavaScript to PHP. They also offer some interesting courses that require minimal commitment and can help take your coding expertise to the next level.

Codrops:

Codrops is a web design and development blog focused on providing its audience with useful tutorials and articles on the latest trends in the industry. The site has a unique approach to content as its very how-to heavy with videos that walk you through processes applicable to all kinds of web developers. Codrops also publishes experimental videos that

focus on new ideas and inspiration for solving problems and tackling challenges around web development. Take a peek and you may walk away with information on how to develop a new skill!

1.5 Company Mission and Vision

Our 60+ team of professionals create beautifully functional web and software solutions from our offices in world wide



CHAPTER: 2

Role and Responsibilities During Internship

2.1 Objective

Main Objective: increase the effectiveness of the bank's telemarketing campaign This project will enable the bank to develop a more granular understanding of its customer base, predict customers' response to its telemarketing campaign and establish a target customer profile for future marketing plans.

By analysing customer features, such as demographics and transaction history, the bank will be able to predict customer saving behaviours and identify which type of customers is more likely to make term deposits. The bank can then focus its marketing efforts on those customers. This will not only allow the bank to secure deposits more effectively but also increase customer satisfaction by reducing undesirable advertisements for certain customers.

2.2 Introduction to Python Programming

Python is widely recognized as a versatile programming language highly valued for its simplicity, clarity, and adaptability, making it a foundational tool in the field of data science.

Its broad applicability spans areas such as statistical analysis, machine learning, and natural language processing, catering to both beginners and experts due to its user-friendly syntax.

Python's extensive standard library offers a wide array of modules and packages tailored to various data-related tasks, including data manipulation, visualization, and machine learning algorithms. Additionally, its support for different programming paradigms, such as procedural, object-oriented, and functional programming, provides data scientists with unmatched flexibility in designing their solutions.

In the realm of data science, Python is complemented by a diverse range of specialized libraries and frameworks, each designed for specific stages of the data analysis process.

For instance, Pandas, NumPy, and Matplotlib excel in data manipulation, numerical computation, and visualization, while scikit-learn and TensorFlow provide comprehensive toolsets for machine learning and deep learning tasks.

Moving beyond libraries, frameworks like Jupyter Notebooks and Apache Spark play crucial roles in the data science landscape. Jupyter Notebooks enable interactive data analysis and visualization, facilitating code experimentation, result visualization, and insight documentation. On the other hand, Apache Spark serves as a distributed computing framework, enabling scalable data processing and analysis across large datasets.

Together, Python and its associated libraries and frameworks form an essential toolkit for data scientists, enabling them to address complex data challenges efficiently and effectively.

2.3 Role and Responsibilities During Internship

"Predicting Customer Response to Telemarketing Campaigns," here are some roles and responsibilities i have taken on during my internship:

1. Data Collection and Preprocessing:

- Gather and clean data from various sources.
- Handle missing values, outliers, and ensure data quality.
- Perform exploratory data analysis (EDA) to understand data patterns and relationships.

2. Feature Engineering:

- Create new features that might improve the predictive power of your models.
- Encode categorical variables and normalize/scale numerical variables as needed.
- Perform feature selection to identify the most important features.

3. Model Building:

- Develop and test different machine learning models such as logistic regression, decision trees, random forests, gradient boosting, etc.
- Fine-tune model parameters using techniques like grid search or randomized search.

4. Model Evaluation:

- Evaluate models using appropriate metrics (e.g., accuracy, precision, recall, F1-score, ROC-AUC).
- Perform cross-validation to ensure model robustness.
- Analyze and compare the performance of different models.

5. Implementation and Deployment:

- Implement the chosen model in a production environment.
- Create APIs or integrate the model with existing systems.
- Ensure the model is scalable and can handle real-time data if necessary.

6. Reporting and Documentation:

- Document the entire process, including data cleaning, feature engineering, model selection, and evaluation.
- Prepare reports and presentations to communicate findings and insights to stakeholders.
- Create dashboards or visualizations to make the results more accessible.

7. Collaboration:

- Work closely with team members, including data engineers, business analysts, and project managers.
- Participate in meetings and provide updates on progress.
- Incorporate feedback from peers and supervisors to improve the project.

8. Continuous Learning:

- Stay updated with the latest trends and techniques in data science and machine learning.
- Apply new methods and tools to improve project outcomes.
- Take part in training sessions or workshops if available.

CHAPTER: 3

Internship Work

3.1 Internship Work

Days	Date	Activities
Day 1	29/06/2024	• Introduction to Data Science • Key Concepts and Terminologies • Conclusion
Day 2	30/06/2024	• Basics of Python Programming • Data Types and Structures • Installing Jupyter Notebook
Day 3	01/07/2024	• Introduction to NumPy Library • Task 1- Generating random numbers using NumPy. • Task 2- Generate array and slicing a NumPy array. • Task 3- Concatenating two NumPy arrays. • Task 4- Sorting a NumPy array
Day 4	02/07/2024	• Introduction to Pandas DataFrame • Task 1- Utilize describe() for statistical summary. • Task 2- Conduct data sorting using sort_values().
Day 5	03/07/2024	• Task 3- Implement hierarchical indexing with MultiIndex. • Task 4- Utilize stack() and unstack() for reshaping data. • Task 5- Apply vectorized operations for efficient computation
Day 6	04/07/2024	• Pandas Library • Task 1: Creating DataFrames with NumPy Arrays • Task 2: DataFrame Operations

		• Task 3: Handling Missing Data • Task 4: Data Aggregation
Day 7	05/07/2024	• Practicing Pandas and NumPy Operations on Raw Data and Learning About CSV Files • Task 1: Loading Raw Data into DataFrame • Task 2: Performing Pandas and NumPy Operations • Task 3: Learning About CSV Files
Day 8	06/07/2024	• Introduction to matplotlib • Creating Basic Plots
Day 9	07/07/2024	• Learning About Seaborn • Task 1: Introduction to Seaborn • Task 2: Creating Basic Plots with Seaborn
Day 10	08/07/2024	• Learning About Pandas Pivot Table, Seaborn Heatmap, and Installing Pandas Profiling • Task 1: Creating Pivot Tables in Pandas • Task 2: Generating Heatmaps with Seaborn • Task 3: Installing and Using Pandas Profiling
Day 11	09/07/2024	• Box Plot • A combined visualization comprising both a box plot and a histogram, providing a comprehensive analysis of data distribution.
Day 12	10/07/2024	• A combined visualization comprising both a heatmap and a scatter plot, providing a comprehensive analysis of data relationships.
Day 13	11/07/2024	• Task 1: Line Plot of Population Trends • Task 2: Distribution of Age Groups • Task 3: Comparison of Population Growth Rates

Day 14	12/07/2024	• Task 1: Line Plot of Time Series Data • Task 2: Swarm Plot of Categorical Data • Task 3: A combined visualization comprising both a line plot and a swarm plot, providing a comprehensive analysis of data trends and distribution.
Day 15	13/07/2024	• Learning About Regression Analysis and Scikit-Learn • Task 1: Introduction to Regression Analysis • Task 2: Simple Linear Regression with Scikit-Learn • Task 3: Multiple Linear Regression

#Day(01)

- **Introduction to Data Science:**

Data Science is an interdisciplinary field that combines techniques from statistics, computer science, and domain expertise to extract meaningful insights from data. It involves processes and methods for understanding, analyzing, and interpreting complex data.

- **Key Concepts and Terminologies:**

1. Data: Raw facts and figures that can be processed to gain information.

- Types of Data:

- Structured Data: Organized in a tabular format (e.g., databases, spreadsheets).
- Unstructured Data: Not organized in a pre-defined manner (e.g., text, images, videos).

2. Data Science Lifecycle:

- Data Collection: Gathering raw data from various sources.
- Data Cleaning: Handling missing values, removing duplicates, and correcting errors.
- Data Exploration: Analysing data to discover patterns and relationships.
- Data Modelling: Building models using statistical and machine learning techniques.
- Data Evaluation: Assessing the performance of models.
- Deployment: Implementing the model in a real-world scenario.
- Monitoring and Maintenance: Ensuring the model performs well over time.

3. Tools and Technologies:

- Programming Languages: Python, R
- Libraries and Frameworks: Pandas, NumPy, Scikit-learn, TensorFlow, Keras
- Data Visualization Tools: Matplotlib, Seaborn, Tableau
- Databases: SQL, NoSQL

4. Key Skills for Data Scientists:

- Statistical Analysis: Understanding and applying statistical methods.
- Machine Learning: Developing algorithms to make predictions or decisions based on data.
- Data Wrangling: Transforming and mapping data from raw form to a more usable format.
- Data Visualization: Creating visual representations of data to communicate findings.
- Domain Knowledge: Understanding the industry and specific problem context.

5. Applications of Data Science:

- Business Intelligence: Improving decision-making processes.
- Healthcare: Predicting disease outbreaks, personalized medicine.
- Finance: Fraud detection, risk management.

- Marketing: Customer segmentation, sentiment analysis.
- Manufacturing: Predictive maintenance, quality control.

- **Conclusion:**

Day 1 provided a foundational understanding of data science, including its key concepts, lifecycle, tools, and skills required. This sets the stage for diving deeper into specific techniques and applications in the following days of the internship.

#Day(02)

- **Basics Of Python Programming:**

Introduction to Python: Python is a high-level, interpreted programming language known for its simplicity and readability. It is widely used in data science due to its extensive libraries and community support.

- **Data Types And Structures:**

Data Types:

Integers: Whole numbers (e.g., `x = 5`)

Floats: Decimal numbers (e.g., `pi = 3.14`)

Strings: Sequence of characters (e.g., `name = "Alice"`)

Booleans: True or False values (e.g., `is_valid = True`)

Data Structures:

Lists: Ordered, mutable collection of items

Python Code

```
fruits = ["apple", "banana", "cherry"]
```

Tuples: Ordered, immutable collection of items

Python Code

```
point = (10, 20)
```

Dictionaries: Unordered collection of key-value pairs

Python Code

```
student = {"name": "Alice", "age": 25}
```

Sets: Unordered collection of unique items

Python Code

```
colors = {"red", "green", "blue"}
```

- **Installing Jupyter Notebook:**

Jupyter Notebook is an open-source web application that allows you to create and share documents containing live code, equations, visualizations, and narrative text. It is a powerful tool for data analysis and visualization in Python.

1. Installing Anaconda:

- Anaconda is a distribution of Python and R for scientific computing and data science. It includes Jupyter Notebook and many other useful packages.

- Steps:
 1. Download the Anaconda installer from the official website.
 2. Run the installer and follow the installation instructions.
 3. Verify the installation by opening a terminal or command prompt and typing:

```
conda --version
```

2. Creating a New Environment (Optional):

- It's a good practice to create a new environment for each project to manage dependencies.
- Steps:

```
conda create --name myenv  
conda activate myenv
```

3. Launching Jupyter Notebook:

- After installing Anaconda, you can launch Jupyter Notebook from the Anaconda Navigator or by using the command line.
- Steps:

```
jupyter notebook
```

- This will open the Jupyter Notebook interface in your default web browser. You can create a new notebook by selecting "New" -> "Python 3".

#Day(03)

- **Introduction to NumPy:** NumPy is essential for numerical computing in Python, providing support for arrays, matrices, and various mathematical functions.
- **Task 1: Generating Random Numbers**

```
[2]: import numpy as np
```

```
# Generate an array of 5 random numbers between 0 and 1  
random_numbers = np.random.rand(5)  
print("Random numbers between 0 and 1:", random_numbers)  
  
# Generate an array of 5 random integers between 10 and 20  
random_integers = np.random.randint(10, 20, size=5)  
print("Random integers between 10 and 20:", random_integers)
```

```
Random numbers between 0 and 1: [0.87354443 0.31717133 0.64261582 0.86309145 0.65321363]  
Random integers between 10 and 20: [18 18 16 10 15]
```

- **Task 2: Array Generation and Slicing**

```
[3]: # Generate a 2D array (3x3)  
array_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  
print("2D Array:\n", array_2d)  
  
# Slicing to get the first two rows and the first two columns  
sliced_array = array_2d[:2, :2]  
print("Sliced Array:\n", sliced_array)
```

```
2D Array:  
[[1 2 3]  
 [4 5 6]  
 [7 8 9]]  
Sliced Array:  
[[1 2]  
 [4 5]]
```

- **Task 3: Concatenating Arrays**

```
[4]: # Create two 1D arrays
array1 = np.array([1, 2, 3])
array2 = np.array([4, 5, 6])

# Concatenate along axis 0 (rows)
concatenated_array = np.concatenate((array1, array2), axis=0)
print("Concatenated Array:", concatenated_array)

# Create two 2D arrays
array2d_1 = np.array([[1, 2], [3, 4]])
array2d_2 = np.array([[5, 6], [7, 8]])

# Concatenate along axis 1 (columns)
concatenated_array2d = np.concatenate((array2d_1, array2d_2), axis=1)
print("Concatenated 2D Array:\n", concatenated_array2d)
```

```
Concatenated Array: [1 2 3 4 5 6]
Concatenated 2D Array:
[[1 2 5 6]
 [3 4 7 8]]
```

- **Task 4: Sorting Arrays**

```
[5]: # Create an unsorted array
unsorted_array = np.array([3, 1, 4, 1, 5, 9, 2, 6, 5])

# Sort the array
sorted_array = np.sort(unsorted_array)
print("Sorted Array:", sorted_array)

# Sort a 2D array along a specific axis
unsorted_array2d = np.array([[3, 2, 1], [6, 5, 4]])
sorted_array2d = np.sort(unsorted_array2d, axis=1)
print("Sorted 2D Array along axis 1:\n", sorted_array2d)
```

```
Sorted Array: [1 1 2 3 4 5 5 6 9]
Sorted 2D Array along axis 1:
[[1 2 3]
 [4 5 6]]
```

- **Conclusion:** Day 3 focused on NumPy, covering generating random numbers, array creation and slicing, concatenation, and sorting. These tasks are fundamental for data manipulation and analysis in data science.

#Day(04)

- **Introduction to Pandas DataFrame:** Pandas is a powerful data manipulation library in Python, offering versatile capabilities for data exploration and analysis through its DataFrame structure.
- **Task 1: Utilize describe() for Statistical Summary**

```
[2]: import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
summary = df.describe()
print(summary)
```

	A	B
count	3.0	3.0
mean	2.0	5.0
std	1.0	1.0
min	1.0	4.0
25%	1.5	4.5
50%	2.0	5.0
75%	2.5	5.5
max	3.0	6.0

- **Task 2: Conduct Data Sorting Using sort_values()**

```
[3]: sorted_df = df.sort_values(by='A')
print(sorted_df)
```

	A	B
0	1	4
1	2	5
2	3	6

#Day(05)

- **Task 3: Implement Hierarchical Indexing with MultiIndex**

```
[4]: arrays = [['a', 'a', 'b', 'b'], [1, 2, 1, 2]]
      index = pd.MultiIndex.from_arrays(arrays, names=('letter', 'number'))
      df_multi = pd.DataFrame({'data': [1, 2, 3, 4]}, index=index)
      print(df_multi)
```

		data
letter	number	
a	1	1
	2	2
b	1	3
	2	4

- **Task 4: Utilize stack() and unstack() for Reshaping Data**

```
[5]: stacked = df_multi.stack()
      print(stacked)
```

letter	number	
a	1	data 1
	2	data 2
b	1	data 3
	2	data 4

dtype: int64

- **Task 5: Apply Vectorized Operations for Efficient Computation**

```
[6]: df['C'] = df['A'] + df['B']
      print(df)
```

	A	B	C
0	1	4	5
1	2	5	7
2	3	6	9

- **Conclusion:** Day 4 & 5 focused on advanced Pandas DataFrame operations, including generating statistical summaries with `describe()`, sorting data with `sort_values()`, implementing hierarchical indexing with `MultiIndex`, reshaping data using `stack()` and `unstack()`, and applying vectorized operations for efficient computation. These tasks enhance the ability to explore and analyse data comprehensively.

#Day(06)

- **Introduction:** The focus of Day 6 was on practicing data manipulation and analysis using Pandas, with support from NumPy for numerical operations.

Task 1: Creating DataFrames with NumPy Arrays

1. Creating DataFrames with NumPy Arrays

```
[1]: import numpy as np
import pandas as pd

np_array = np.random.rand(5, 3)
df_from_numpy = pd.DataFrame(np_array, columns=['A', 'B', 'C'])
print(df_from_numpy)
```

	A	B	C
0	0.410508	0.038377	0.402270
1	0.398641	0.322544	0.417674
2	0.699677	0.163306	0.292562
3	0.170387	0.882938	0.758661
4	0.942883	0.942343	0.446045

Task 2: DataFrame Operations

2. Calculate mean of each column:

```
[2]: means = df_from_numpy.mean()
print(means)
```

```
A    0.524419
B    0.469902
C    0.463443
dtype: float64
```

3. Add a new column with vectorized operations:

```
[3]: df_from_numpy['D'] = df_from_numpy['A'] + df_from_numpy['B']
print(df_from_numpy)
```

	A	B	C	D
0	0.410508	0.038377	0.402270	0.448886
1	0.398641	0.322544	0.417674	0.721184
2	0.699677	0.163306	0.292562	0.862983
3	0.170387	0.882938	0.758661	1.053324
4	0.942883	0.942343	0.446045	1.885226

Task 3: Handling Missing Data

Introduce missing values and handle them:

```
[4]: df_from_numpy.iloc[2, 1] = np.nan
      print(df_from_numpy)
```

	A	B	C	D
0	0.410508	0.038377	0.402270	0.448886
1	0.398641	0.322544	0.417674	0.721184
2	0.699677	NaN	0.292562	0.862983
3	0.170387	0.882938	0.758661	1.053324
4	0.942883	0.942343	0.446045	1.885226

Fill missing values with column mean:

```
[5]: df_from_numpy['B'].fillna(df_from_numpy['B'].mean(), inplace=True)
      print(df_from_numpy)
```

	A	B	C	D
0	0.410508	0.038377	0.402270	0.448886
1	0.398641	0.322544	0.417674	0.721184
2	0.699677	0.546551	0.292562	0.862983
3	0.170387	0.882938	0.758661	1.053324
4	0.942883	0.942343	0.446045	1.885226

Task 4: Data Aggregation

Group by and calculate sum:

```
[6]: df_from_numpy['Category'] = ['A', 'A', 'B', 'B', 'C']
      grouped = df_from_numpy.groupby('Category').sum()
      print(grouped)
```

	A	B	C	D
Category				
A	0.809149	0.360921	0.819945	1.170070
B	0.870064	1.429489	1.051223	1.916308
C	0.942883	0.942343	0.446045	1.885226

Conclusion: Day 6 was dedicated to practicing data manipulation and analysis using Pandas and NumPy. Tasks included creating DataFrames from NumPy arrays, performing DataFrame operations, handling missing data, and aggregating data. These exercises enhanced skills in handling and analysing data effectively.

#Day(07)

- **Introduction:** The focus of Day 7 was on practicing data manipulation and analysis using Pandas and NumPy on raw data. Additionally, the day included learning about handling CSV files for data storage and retrieval.
- **Task 1: Loading Raw Data into DataFrame**

Load data into a Pandas DataFrame:

```
[1]: import pandas as pd

data = {
    'ID': [1, 2, 3, 4, 5],
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [30, 25, 35, 28, 40],
    'Salary': [70000, 50000, 120000, 60000, 150000],
    'Department': ['HR', 'IT', 'Finance', 'IT', 'Management']
}
df = pd.DataFrame(data)
print(df)
```

	ID	Name	Age	Salary	Department
0	1	Alice	30	70000	HR
1	2	Bob	25	50000	IT
2	3	Charlie	35	120000	Finance
3	4	David	28	60000	IT
4	5	Eve	40	150000	Management

- **Task 2: Performing Pandas and NumPy Operations**

Calculate average salary:

```
[2]: avg_salary = df['Salary'].mean()
print(f"Average Salary: {avg_salary}")
```

Average Salary: 90000.0

Filter employees with salary above 80000:

```
[4]: high_salary_df = df[df['Salary'] > 80000]
print(high_salary_df)
```

	ID	Name	Age	Salary	Department
2	3	Charlie	35	120000	Finance
4	5	Eve	40	150000	Management

- **Task 3: Learning About CSV Files**

Save DataFrame to CSV file:

```
[5]: df.to_csv('employees.csv', index=False)
```

Load DataFrame from CSV file:

```
[6]: loaded_df = pd.read_csv('employees.csv')
print(loaded_df)
```

	ID	Name	Age	Salary	Department
0	1	Alice	30	70000	HR
1	2	Bob	25	50000	IT
2	3	Charlie	35	120000	Finance
3	4	David	28	60000	IT
4	5	Eve	40	150000	Management

- **Conclusion:** Day 7 was dedicated to practicing data manipulation and analysis on raw data using Pandas and NumPy. Additionally, it covered learning about handling CSV files for data storage and retrieval. These exercises reinforced the ability to effectively work with real-world data.

#Day(08)

- **Introduction:** The focus of Day 8 was on learning about Matplotlib, a powerful plotting library in Python used for creating static, animated, and interactive visualizations.

- **Task 1: Introduction to Matplotlib**

Understand the basics of Matplotlib and its importance in data visualization. Learn about key components: Figure, Axes, and Plot.

- **Task 2: Creating Basic Plot**

Line Plot, Scatter Plot, Bar Plot, Histogram etc.

- **Conclusion:** Day 8 was dedicated to learning about Matplotlib and its capabilities for data visualization. Tasks included creating basic plots such as line plots, scatter plots, bar plots, and histograms, along with customizing plots for better clarity and presentation. These skills are essential for visualizing data effectively in data science and machine learning projects.

#Day(09)

- **Introduction:** The focus of Day 9 was on learning about Seaborn, a statistical data visualization library built on top of Matplotlib. Seaborn provides a high-level interface for drawing attractive and informative statistical graphics.

- **Tasks:**

1. Introduction to Seaborn

Understand the basics of Seaborn and its importance in enhancing Matplotlib visualizations.

Learn about key features and datasets provided by Seaborn.

2. Creating basic plots with seaborn

- **Scatter Plot:**

Creating Basic Plots with Seaborn

Scatter Plot

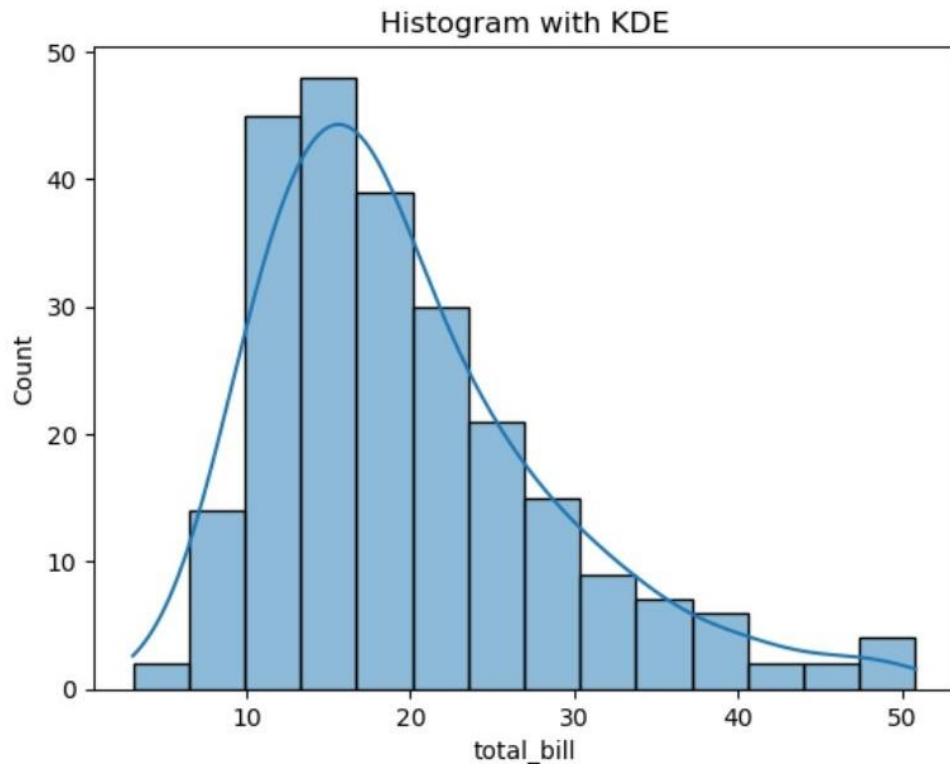
```
[1]: import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset('tips')
sns.scatterplot(x='total_bill', y='tip', data=tips)
plt.title("Scatter Plot")
plt.show()
```

- **Histogram and KDE Plots:**

Create a histogram with a KDE (Kernel Density Estimate) plot

```
[3]: sns.histplot(tips['total_bill'], kde=True)
plt.title("Histogram with KDE")
plt.show()
```



- **Conclusion:** Day 9 was dedicated to learning about Seaborn and its capabilities for statistical data visualization. Tasks included creating various types of plots such as scatter plots, histograms with KDE, bar plots, box plots, pair plots, and heatmaps. Seaborn enhances the visualization process by making it easier to create informative and attractive statistical graphics.

#Day(10)

- **Introduction:** The focus of Day 10 was on creating pivot tables in Pandas, generating heatmaps in Seaborn, and installing Pandas Profiling for comprehensive data analysis.
- **Task 1:** Creating Pivot Tables in Pandas

Creating Pivot Tables in Pandas

```
[1]: import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Department': ['HR', 'IT', 'Finance', 'IT', 'HR'],
    'Salary': [70000, 50000, 120000, 60000, 150000]
}
df = pd.DataFrame(data)

pivot_table = df.pivot_table(values='Salary', index='Department', aggfunc='mean')
print(pivot_table)
```

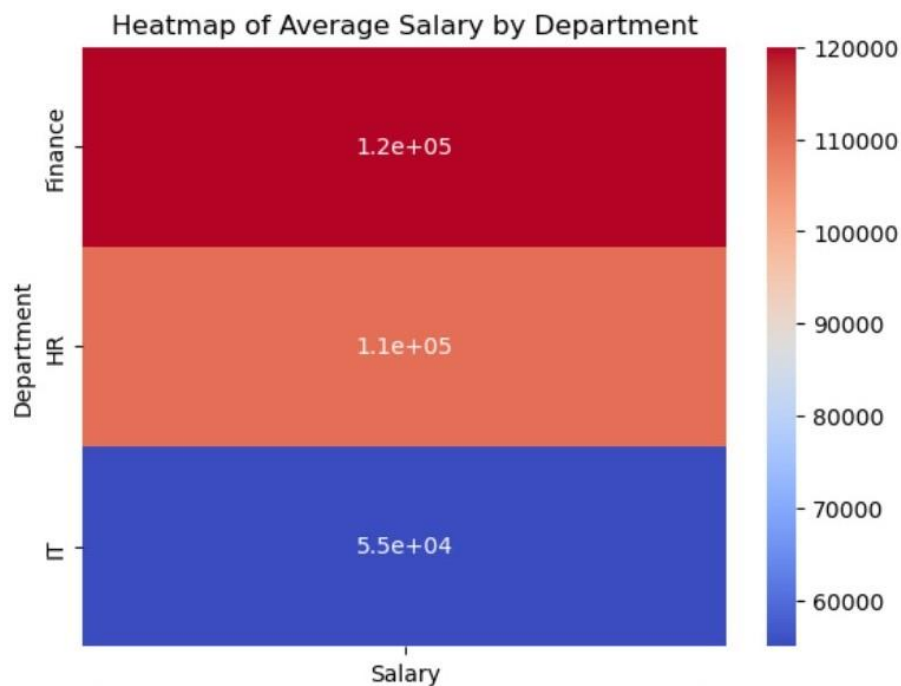
	Salary
Department	
Finance	120000.0
HR	110000.0
IT	55000.0

- **Task 2:** Generating Heatmaps with Seaborn

Generating Heatmaps with Seaborn

```
[2]: import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(pivot_table, annot=True, cmap='coolwarm')
plt.title("Heatmap of Average Salary by Department")
plt.show()
```



- **Task 3:** Installing and Using Pandas Profiling

```
[7]: from pandas_profiling import ProfileReport

profile = ProfileReport(df, title='Pandas Profiling Report', explorative=True)
profile.to_file("pandas_profiling_report.html")
```

- **Conclusion:** Day 10 focused on advanced data summarization and visualization techniques. Tasks included creating pivot tables in Pandas to summarize data, generating heatmaps in Seaborn to visualize summarized data, and installing Pandas Profiling to generate comprehensive data analysis reports. These tools are essential for gaining deeper insights and presenting data effectively.

#Day(11)

- **Task 1: Box Plot**

Source Code:

```
import matplotlib.pyplot as plt

import numpy as np

# Generate sample data

np.random.seed(0)

data = np.random.normal(loc=0, scale=1, size=100)

# Create a box plot

plt.boxplot(data)

plt.title('Box Plot: Data Distribution')

plt.ylabel('Values')

plt.show()
```

Explanation: We import Matplotlib as plt and NumPy as np.

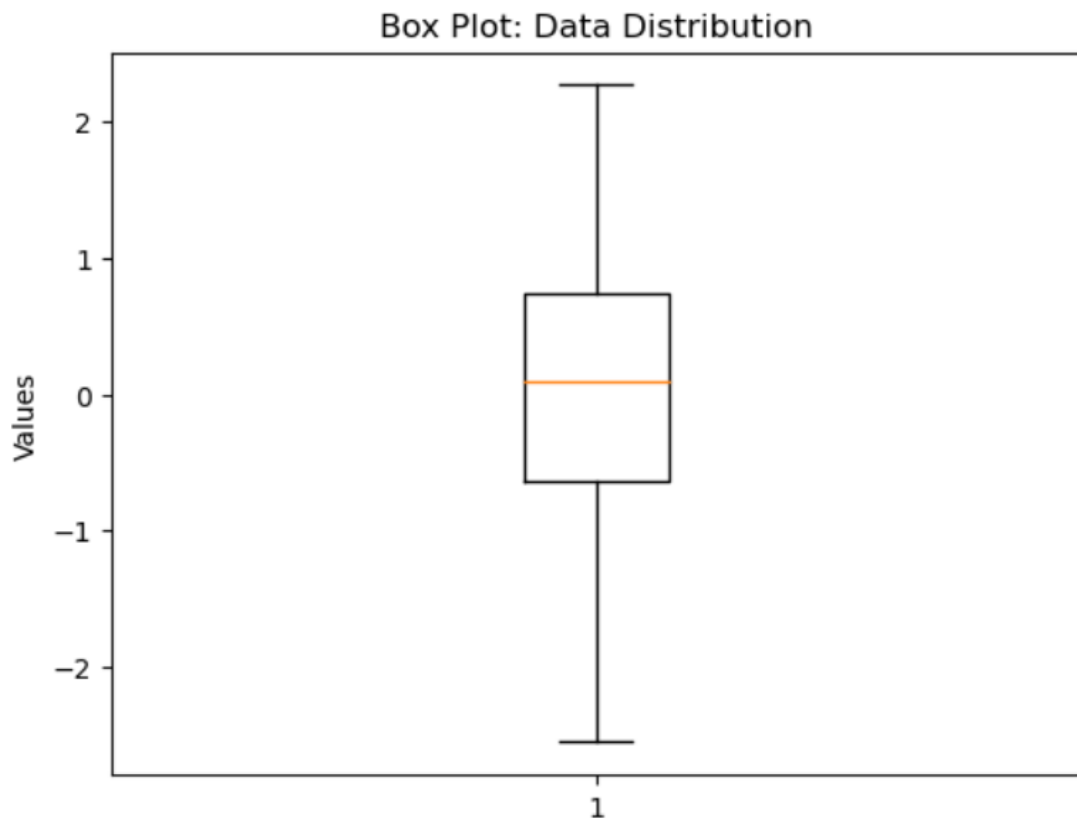
Sample data is generated using NumPy's **random.normal()** function.

A box plot is created using Matplotlib's **boxplot()** function.

Title and labels are added to the plot.

The box plot is displayed using **show()**.

- **Output:**



- **Task 2:** A combined visualization comprising both a box plot and a histogram, providing a comprehensive analysis of data distribution.

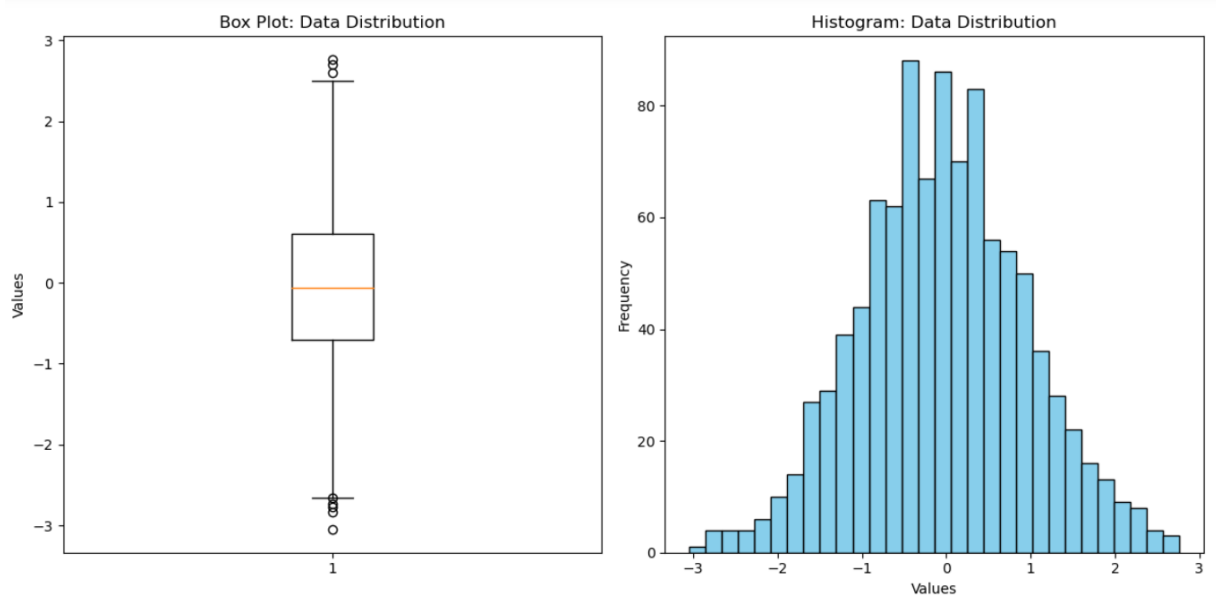
- **Source code:**

```
import matplotlib.pyplot as plt
import numpy as np
# Generate sample data
np.random.seed(0)
data = np.random.normal(loc=0, scale=1, size=1000)
# Create subplots for box plot and histogram
fig, axs = plt.subplots(1, 2, figsize=(12, 6))
# Box plot
axs[0].boxplot(data)
axs[0].set_title('Box Plot: Data Distribution')
axs[0].set_ylabel('Values')
# Histogram
axs[1].hist(data, bins=30, color='skyblue', edgecolor='black')
axs[1].set_title('Histogram: Data Distribution')
axs[1].set_xlabel('Values')
axs[1].set_ylabel('Frequency')
# Adjust layout
plt.tight_layout()
plt.show()
```

- **Explanation:**

1. We import Matplotlib as plt and NumPy as np.
2. Sample data is generated using NumPy's **random.normal()** function.
3. Subplots are created to accommodate both the box plot and histogram.
4. For the box plot, **boxplot()** function is used, and for the histogram, **hist()** function is used.
5. Titles, labels, and customizations are applied to both plots.
6. **tight_layout()** ensures optimal spacing between subplots.
7. The combined visualization is displayed using **show()**.

- **Output:**



#Day(12)

- **Task:** A combined visualization comprising both a heatmap and a scatter plot, providing a comprehensive analysis of data relationships.

- **Source Code:**

```
import matplotlib.pyplot as plt

import numpy as np

# Generate sample data

np.random.seed(0)

data = np.random.rand(10, 10)

x = np.random.rand(100)

y = np.random.rand(100)

# Create subplots for heatmap and scatter plot

fig, axs = plt.subplots(1, 2, figsize=(12, 6))

# Heatmap

axs[0].imshow(data, cmap='hot', interpolation='nearest')

axs[0].set_title('Heatmap: Correlation Matrix')

axs[0].set_xlabel('X-axis')

axs[0].set_ylabel('Y-axis')

axs[0].set_xticks([])

axs[0].set_yticks([])

axs[0].grid(False)

# Scatter plot

axs[1].scatter(x, y, color='skyblue', alpha=0.7)

axs[1].set_title('Scatter Plot: Relationship Between Variables')
```

```
axs[1].set_xlabel('X-axis')
```

```
axs[1].set_ylabel('Y-axis')
```

```
# Adjust layout
```

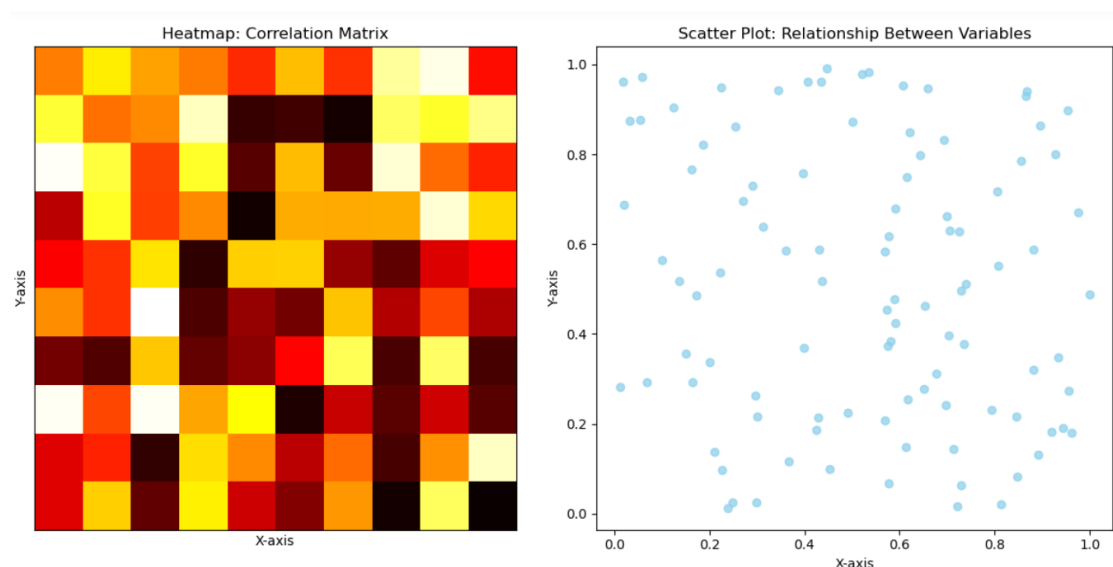
```
plt.tight_layout()
```

```
plt.show()
```

- **Explanation:**

1. We import Matplotlib as plt and NumPy as np.
2. Sample data is generated for both a correlation matrix and x, y coordinates.
3. Subplots are created to accommodate both the heatmap and scatter plot.
4. For the heatmap, imshow() function is used with options for colormap and interpolation.
5. For the scatter plot, scatter() function is used with options for color and transparency.
6. Titles, labels, and customizations are applied to both plots.
7. tight_layout() ensures optimal spacing between subplots.
8. The combined visualization is displayed using show().

- **Output:**



#Day(13)

- **Task 1:** Line Plot of Population Trends
- **Source Code:**

```
import seaborn as sns

import matplotlib.pyplot as plt

# Sample data: Year and Population

years = [2010, 2015, 2020, 2025, 2030]

population = [100, 120, 150, 180, 200]

# Create a line plot

sns.lineplot(x=years, y=population)

plt.title('Population Trends Over Time')

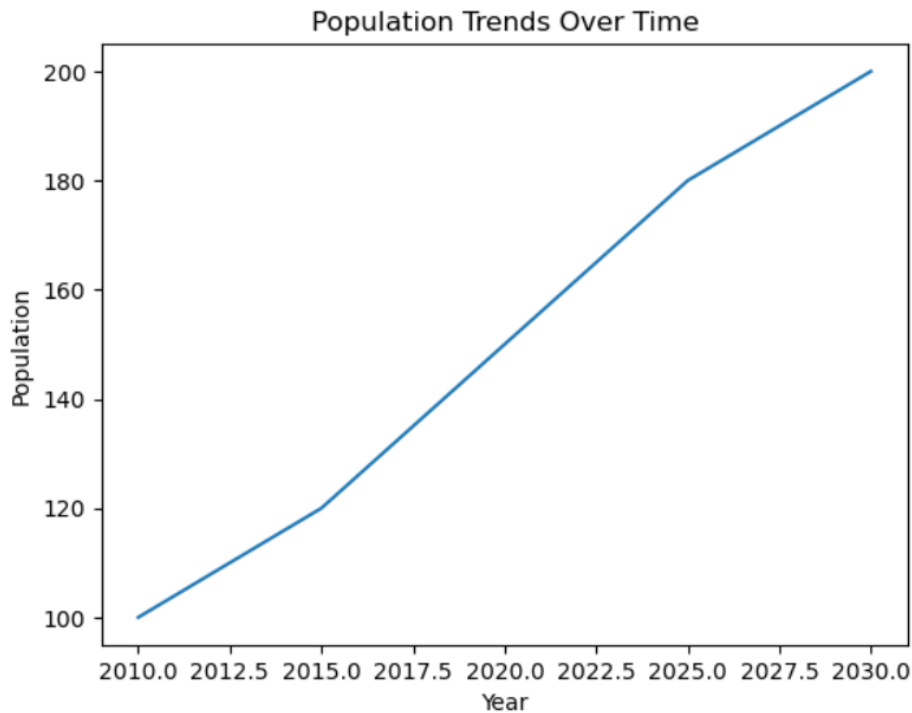
plt.xlabel('Year')

plt.ylabel('Population')

plt.show()
```

- **Explanation:**
 1. We import Seaborn as sns and Matplotlib as plt.
 2. Sample data for years and corresponding population sizes are defined.
 3. A line plot is created using Seaborn's **lineplot()** function, specifying years on the x-axis and population on the y-axis.
 4. Title, labels, and customization options are added to the plot.
 5. The line plot is displayed using show().

- **Output:**



- **Task 2:** Distribution of Age Groups
- **Source Code:**

```
import seaborn as sns

import matplotlib.pyplot as plt

# Sample data: Age distribution
ages = [20, 25, 30, 35, 40, 45, 50, 55, 60]

population_distribution = [1000, 1500, 2000, 1800, 1600, 1400, 1200, 1000, 800]

# Create a bar plot

sns.barplot(x=ages, y=population_distribution, palette='viridis')

plt.title('Distribution of Age Groups')

plt.xlabel('Age')

plt.ylabel('Population')

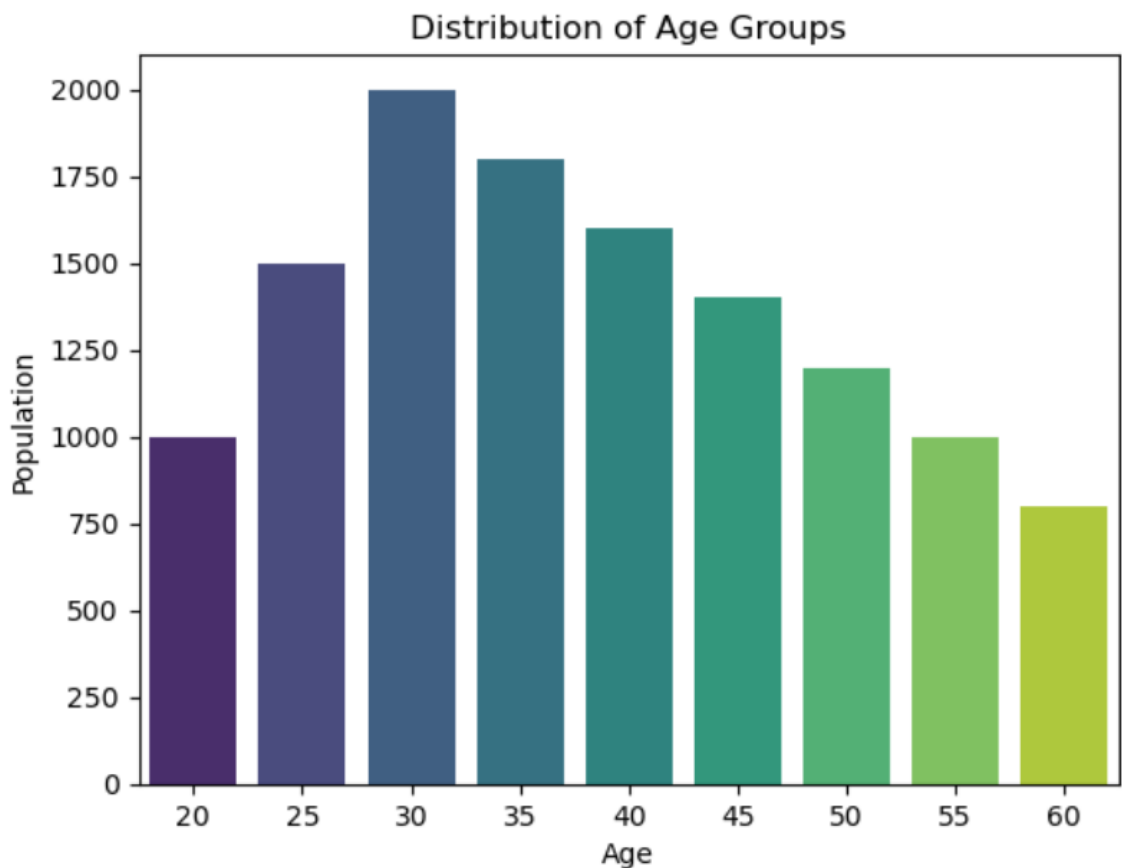
plt.show()
```

- **Explanation:**

1. We import Seaborn as sns and Matplotlib as plt.

2. Sample data for age groups and their corresponding population distribution are defined.
3. A bar plot is created using Seaborn's `barplot()` function, specifying age groups on the x-axis and population distribution on the y-axis.

- **Output:**



- **Task 3:** Comparison of Population Growth Rates
- **Source Code:**

```
import seaborn as sns

import matplotlib.pyplot as plt

# Sample data: Population growth rates by region
regions = ['East', 'West', 'North', 'South']

growth_rates = [3.5, 2.8, 4.2, 3.0]

# Create a horizontal bar plot
```

```
sns.barplot(x=growth_rates, y=regions, palette='rocket', orient='h')

plt.title('Population Growth Rates by Region')

plt.xlabel('Growth Rate (%)')

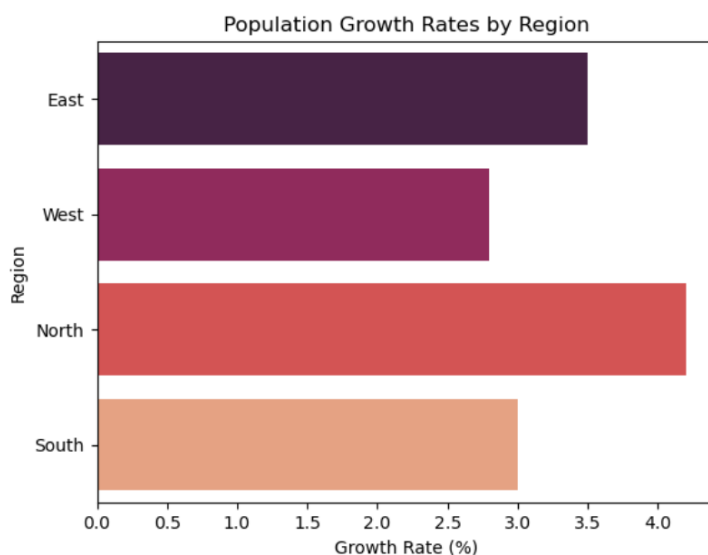
plt.ylabel('Region')

plt.show()
```

- **Explanation:**

1. We import Seaborn as sns and Matplotlib as plt.
2. Sample data for regions and their corresponding population growth rates are defined.
3. A horizontal bar plot is created using Seaborn's barplot() function, specifying growth rates on the x-axis and regions on the y-axis.
4. Color palette ('rocket') and orientation ('h') are specified for aesthetic purposes.
5. Title, labels, and customization options are added to the plot.
6. The bar plot is displayed using show().

- **Output:**



#Day(14)

- **Task 1:** Line Plot of Time Series Data
- **Source Code:**

```
import seaborn as sns

import matplotlib.pyplot as plt

# Sample time series data

time = [1, 2, 3, 4, 5]

values = [10, 15, 12, 18, 20]

# Create a line plot

sns.lineplot(x=time, y=values)

plt.title('Line Plot: Time Series Data')

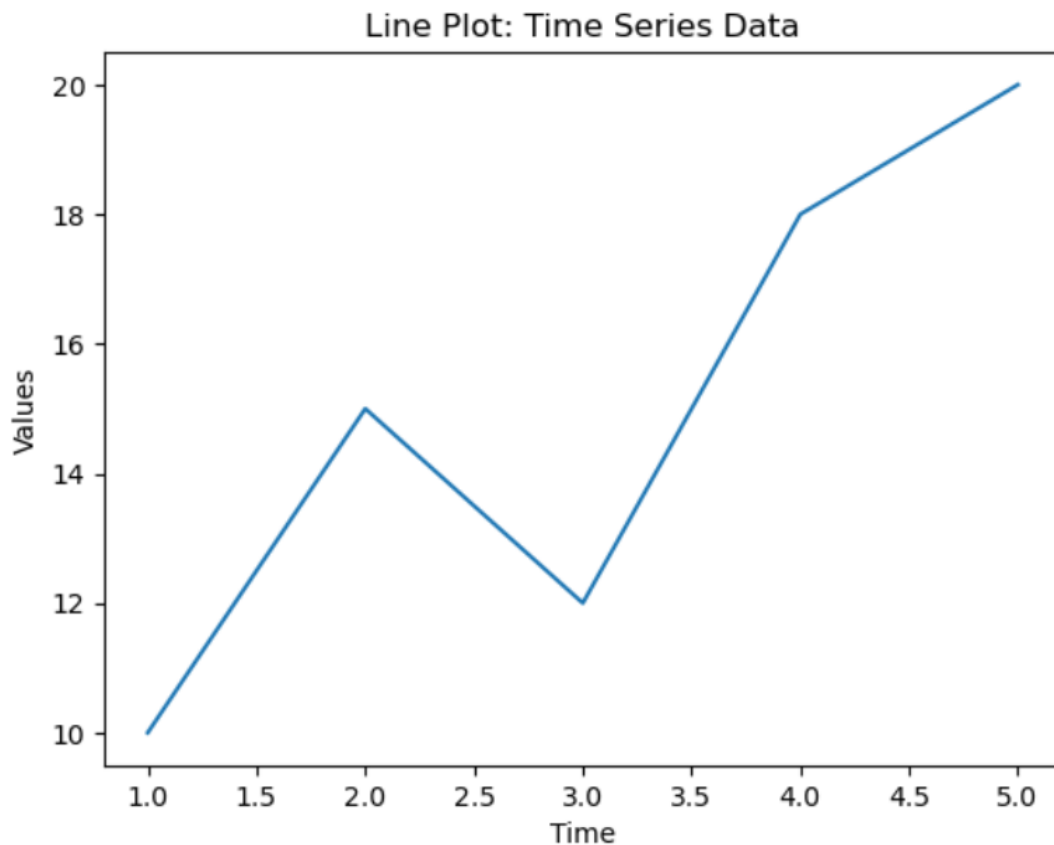
plt.xlabel('Time')

plt.ylabel('Values')

plt.show()
```

- **Explanation:**
 1. We import Seaborn as sns and Matplotlib as plt.
 2. Sample time series data is defined with time points and corresponding values.
 3. A line plot is created using Seaborn's lineplot() function, specifying time on the x-axis and values on the y-axis.
 4. Title, labels, and customization options are added to the plot.
 5. The line plot is displayed using show().

- **Output:**



- **Task 2:** Swarm Plot of Categorical Data
- **Source Code:**

```
import seaborn as sns

import matplotlib.pyplot as plt

# Sample categorical data
categories = ['A', 'B', 'A', 'C', 'B', 'B', 'C', 'A', 'C']

# Create a swarm plot
sns.swarmplot(x=categories)

plt.title('Swarm Plot: Categorical Data Distribution')

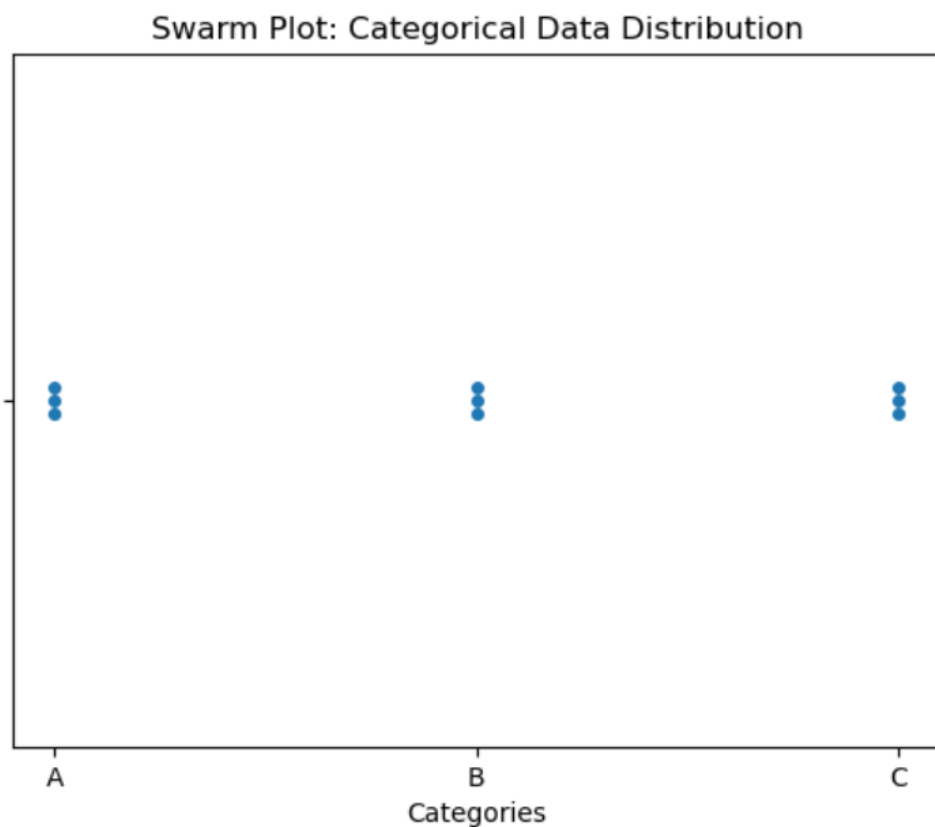
plt.xlabel('Categories')

plt.show()
```

- **Explanation:**

1. We import Seaborn as sns and Matplotlib as plt.
2. Sample categorical data is defined.
3. A swarm plot is created using Seaborn's swarmplot() function, specifying categories on the x-axis.
4. Title, labels, and customization options are added to the plot.
5. The swarm plot is displayed using show().

- **Output:**



- **Task 3:** Combined Visualization
- **Source Code:**

```
import seaborn as sns

import matplotlib.pyplot as plt

# Sample time series data

time = [1, 2, 3, 4, 5]
```

```

values = [10, 15, 12, 18, 20]

# Sample categorical data

categories = ['A', 'B', 'A', 'C', 'B', 'B', 'C', 'A', 'C']

# Create subplots for line plot and swarm plot

fig, axs = plt.subplots(1, 2, figsize=(12, 6))

# Line plot

sns.lineplot(x=time, y=values, ax=axs[0])

axs[0].set_title('Line Plot: Time Series Data')

axs[0].set_xlabel('Time')

axs[0].set_ylabel('Values')

# Swarm plot

sns.swarmplot(x=categories, ax=axs[1])

axs[1].set_title('Swarm Plot: Categorical Data Distribution')

axs[1].set_xlabel('Categories')

# Adjust layout

plt.tight_layout()

plt.show()

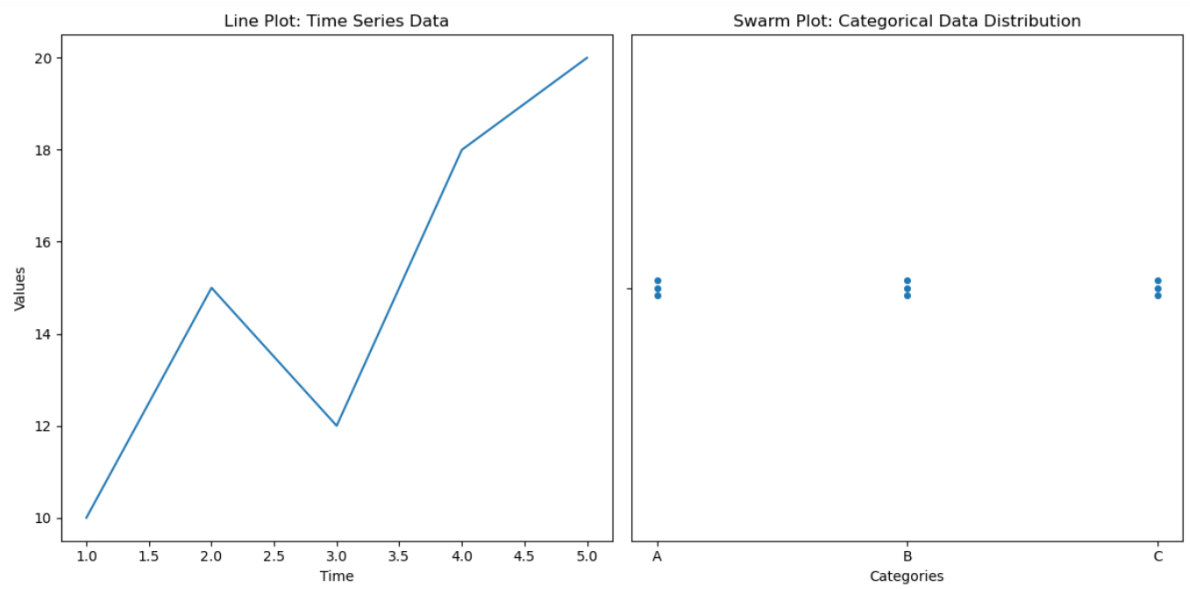
```

- **Explanation:**

1. We import Seaborn as sns and Matplotlib as plt.
2. Sample time series data and categorical data are defined.
3. Subplots are created to accommodate both the line plot and swarm plot.
4. For the line plot, lineplot() function is used, and for the swarm plot, swarmplot() function is used.
5. Titles, labels, and customization options are applied to both plots.
6. tight_layout() ensures optimal spacing between subplots.

7. The combined visualization is displayed using show().

- **Output:**



#Day(15)

- **Introduction:** The focus of the final day was on understanding regression analysis and utilizing scikit-learn, a powerful machine learning library in Python, for implementing regression models.
- **Task 1:** Introduction to Regression Analysis

Introduction to Regression Analysis

Understand the concept of regression analysis and its importance in predictive modeling.

Learn about different types of regression, including linear regression and multiple regression.

- **Task 2:** Simple Linear Regression with Scikit-Learn

Import necessary libraries and load dataset:

```
[1]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Sample dataset
data = {
    'Experience': [1, 2, 3, 4, 5],
    'Salary': [30000, 35000, 50000, 60000, 70000]
}
df = pd.DataFrame(data)

# Split the data into training and testing sets
X = df[['Experience']]
y = df['Salary']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Train a linear regression model:

```
[2]: model = LinearRegression()  
      model.fit(X_train, y_train)
```

```
[2]: ▼ LinearRegression ⓘ ?  
      LinearRegression()
```

Predict and evaluate the model:

```
[3]: y_pred = model.predict(X_test)  
      mse = mean_squared_error(y_test, y_pred)  
      print(f"Mean Squared Error: {mse}")  
      print(f"Model Coefficient: {model.coef_}")  
      print(f"Model Intercept: {model.intercept_}")
```

```
Mean Squared Error: 25000000.0  
Model Coefficient: [10000.]  
Model Intercept: 20000.000000000007
```

- **Task 3: Multiple Linear Regression**

Extend the model to include multiple features

```
[4]: # Sample dataset with multiple features
data = {
    'Experience': [1, 2, 3, 4, 5],
    'Education': [3, 4, 2, 5, 4],
    'Salary': [30000, 35000, 50000, 60000, 70000]
}
df = pd.DataFrame(data)

# Split the data into training and testing sets
X = df[['Experience', 'Education']]
y = df['Salary']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train a multiple linear regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict and evaluate the model
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error: {mse}")
print(f"Model Coefficients: {model.coef_}")
print(f"Model Intercept: {model.intercept_}")

Mean Squared Error: 25000000.0
Model Coefficients: [1.00000000e+04 1.84479196e-12]
Model Intercept: 19999.999999999993
```

- **Conclusion:** The final day of the internship focused on regression analysis and using scikit-learn for implementing regression models. Tasks included training and evaluating simple linear regression and multiple linear regression models. These skills are crucial for building predictive models and performing data analysis in machine learning projects.

Project Code

Load the raw data: This dataset is about the direct phone call marketing campaigns, which aim to promote term deposits among existing customers, by a Portuguese banking institution from May 2008 to November 2010. It is publicly available in the UCI Machine learning Repository, which can be retrieved from <http://archive.ics.uci.edu/ml/datasets/Bank+Marketing#>.

```
[1] import pandas as pd
import numpy as np
from pandas import read_csv
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
import os

linkname = 'bank-full.csv'
dataset1 = pd.read_csv(linkname, sep = ';')
```

There are 41,188 observations in this dataset. Each represents an existing customer that the bank reached via phone calls.

- For each observation, the dataset records **16 input variables** that stand for both qualitative and quantitative attributes of the customer, such as age, job, housing and personal loan status, account balance, and the number of contacts.
- There is a **single binary output variable** that denotes "yes" or "no" revealing the outcomes of the phone calls.

```
[2] # View the first 5 rows in the dataset
dataset1.head()
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	58	management	married	tertiary	no	2143	yes	no	unknown	5	may	261	1	-1	0	unknown	no
1	44	technician	single	secondary	no	29	yes	no	unknown	5	may	151	1	-1	0	unknown	no
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5	may	76	1	-1	0	unknown	no
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	5	may	92	1	-1	0	unknown	no
4	33	unknown	single	unknown	no	1	no	no	unknown	5	may	198	1	-1	0	unknown	no

Clean the Dataset:

Deal with missing data

```
Clean the dataset

2.1 Deal with missing data

There is no missing value in this dataset. Nevertheless, there are values like "unknown", "others", which are helpless just like missing values. Thus, these ambiguous values are removed from the dataset.
```

```
[3] # Step 1: Delete the rows which column 'poutcome' contains 'other'
condition = dataset1.poutcome == 'other'
dataset2 = dataset1.drop(dataset1[condition].index, axis = 0, inplace = False)
```

```
[4] # Step 2: Replace 'unknown' in job and education with 'other'
dataset2[['job', 'education']] = dataset2[['job', 'education']].replace(['unknown'], 'other')
```

+ Code + Markdown

Drop outliers in the column 'balance'

```
2.2 Drop outliers in the column 'balance'
```

In order to capture the general trend in the dataset, outliers in the column "balance" are dropped. Outliers are defined as the values which are more than three standard deviations away from the mean. In sum, 2556 rows of data were removed.

```
from scipy.stats import zscore

dataset2[['balance']].mean()
dataset2[['balance']].mean()

dataset2['balance_outliers'] = dataset2['balance']
dataset2['balance_outliers'] = zscore(dataset2['balance_outliers'])

condition1 = (dataset2['balance_outliers'] > 3) | (dataset2['balance_outliers'] < -3)
dataset3 = dataset2.drop(dataset2[condition1].index, axis = 0, inplace = False)

dataset4 = dataset3.drop('balance_outliers', axis=1)
```

Creating and transforming data

```
Predicting Customer Response to Telemarketing Campaigns.ipynb
```

D: > Sem 7 > New > Predicting Customer Response to Telemarketing Campaigns.ipynb > Part 2. Data Cleaning > Clean the dataset > 2.2 Drop outliers in the column 'balance'

2.3 Creating and transforming data

Some changes were made to the column name, units and data types for easier analysis.

```
# Step 1: Change column name: 'y' to 'response'
dataset4.rename(index=str, columns={'y': 'response'}, inplace = True)

def convert(dataset4, new_column, old_column):
    dataset4[new_column] = dataset4[old_column].apply(lambda x: 0 if x == 'no' else 1)
    return dataset4[new_column].value_counts()

convert(dataset4, "response_binary", "response")

response_binary
0    37785
1     4870
Name: count, dtype: int64

# Step 2: Drop column "contact" which is useless
dataset5 = dataset4.drop("contact", axis=1)

# Step 3: Change the unit of 'duration' from seconds to minutes
dataset5['duration'] = dataset5['duration'].apply(lambda x: x/60).round(2)

# Step 4: Change 'month' from words to numbers for easier analysis
lst = [dataset5]
for column in lst:
```

Filtering

```
Predicting Customer Response to Telemarketing Campaigns.ipynb
D: > Sem 7 > New > Predicting Customer Response to Telemarketing Campaigns.ipynb > M4 Part 2. Data Cleaning > M4 2.2 Drop outliers in the column 'balance'
+ Code + Markdown | Run All | Restart | Clear All Outputs | Variables | Outline ... Python 3.12.4

column.loc[column["month"] == "jun", "month_int"] = 6
column.loc[column["month"] == "jul", "month_int"] = 7
column.loc[column["month"] == "aug", "month_int"] = 8
column.loc[column["month"] == "sep", "month_int"] = 9
column.loc[column["month"] == "oct", "month_int"] = 10
column.loc[column["month"] == "nov", "month_int"] = 11
column.loc[column["month"] == "dec", "month_int"] = 12

[10] ✓ 0.0s Python

2.4 Filtering

# Step 1: Drop rows that 'duration' < 5s
condition2 = (dataset5['duration'] < 5/60)
dataset6 = dataset5.drop(dataset5[condition2].index, axis = 0, inplace = False)

[11] ✓ 0.0s Python

# Step 2: Drop customer values with 'other' education
condition3 = (dataset6['education'] == 'other')
dataset7 = dataset6.drop(dataset6[condition3].index, axis = 0, inplace = False)

[12] ✓ 0.0s Python

Part 3. Exploratory Data Analysis

To obtain a better understanding of the dataset, the distribution of key variables and the relationships among them were plotted.

[Redacted]
```

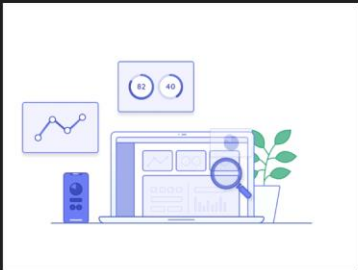
Exploratory Data Analysis

```
Predicting Customer Response to Telemarketing Campaigns.ipynb
D: > Sem 7 > New > Predicting Customer Response to Telemarketing Campaigns.ipynb > M4 Part 2. Data Cleaning > M4 Clean the dataset > M4 2.2 Drop outliers in the column 'balance'
+ Code + Markdown | Run All | Restart | Clear All Outputs | Variables | Outline ... Python 3.12.4

[12] ✓ 0.0s Python

Part 3. Exploratory Data Analysis

To obtain a better understanding of the dataset, the distribution of key variables and the relationships among them were plotted.

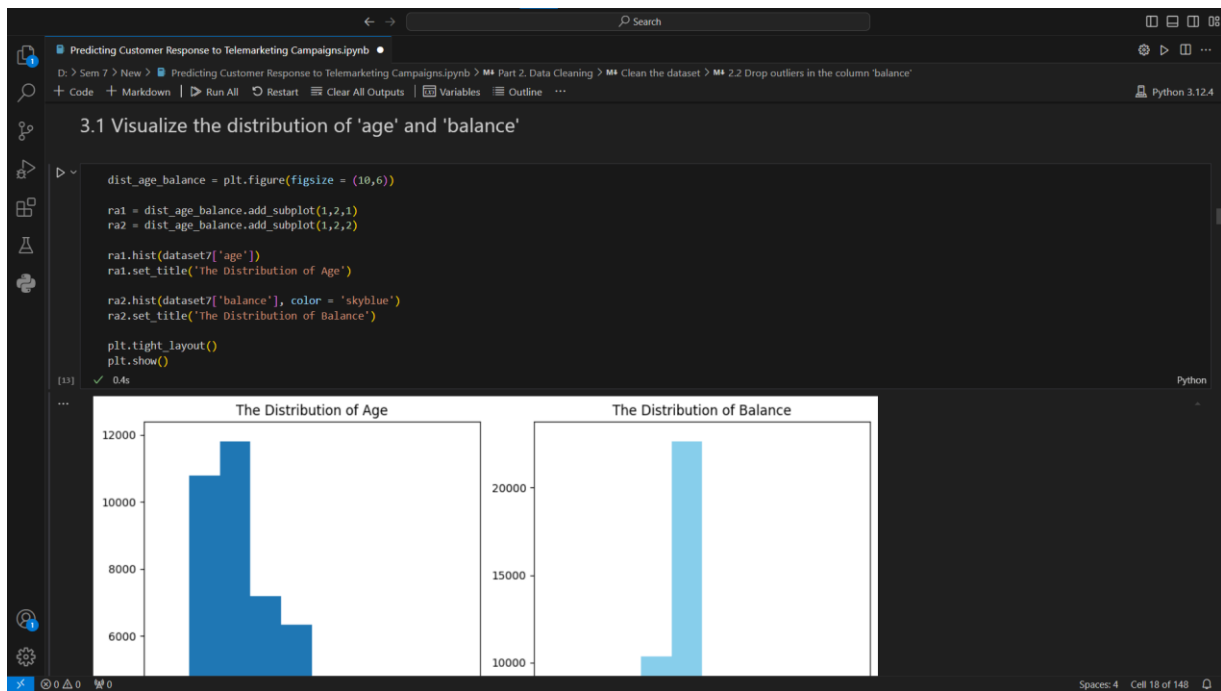


3.1 Visualize the distribution of 'age' and 'balance'

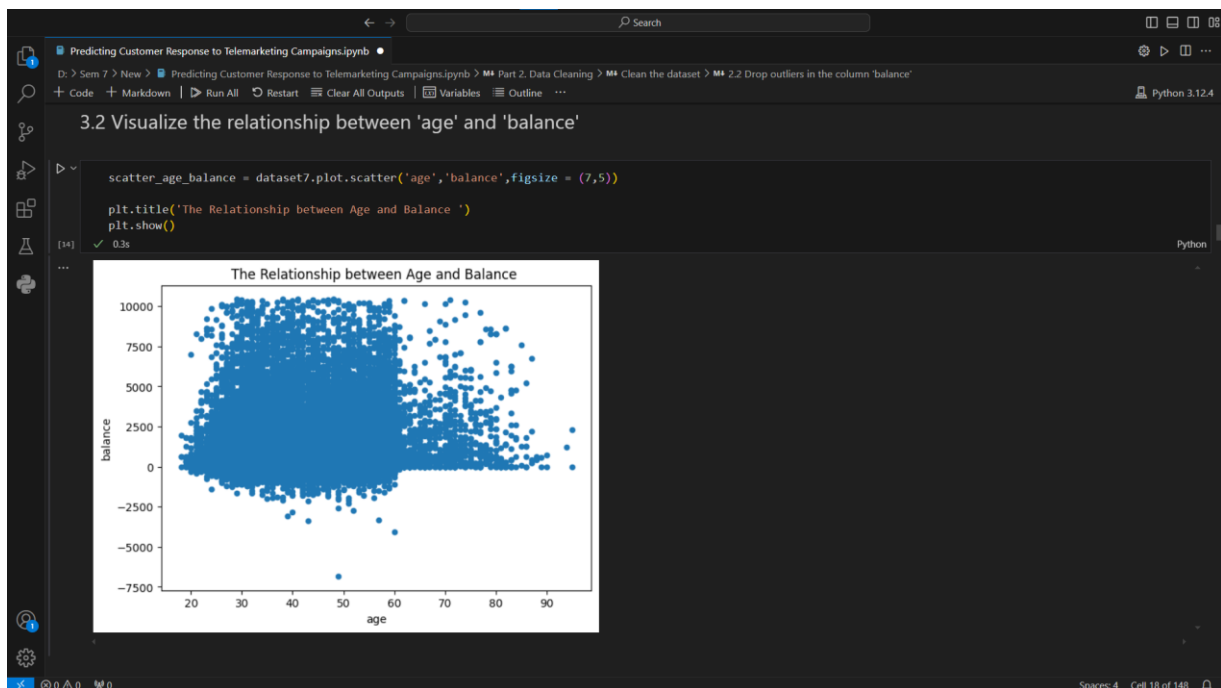
dist_age_balance = plt.figure(figsize = (10,6))

ra1 = dist_age_balance.add_subplot(1,2,1)
ra2 = dist_age_balance.add_subplot(1,2,2)
```

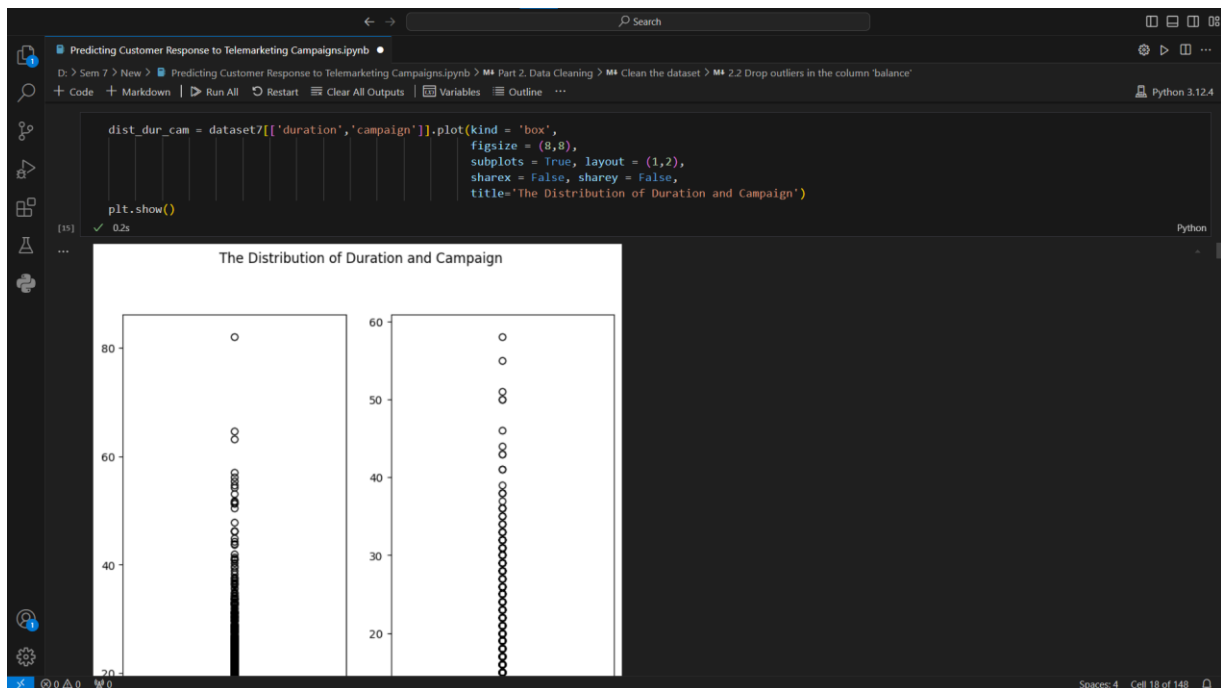
Visualize the distribution of 'age' and 'balance'



Visualize the relationship between 'age' and 'balance'

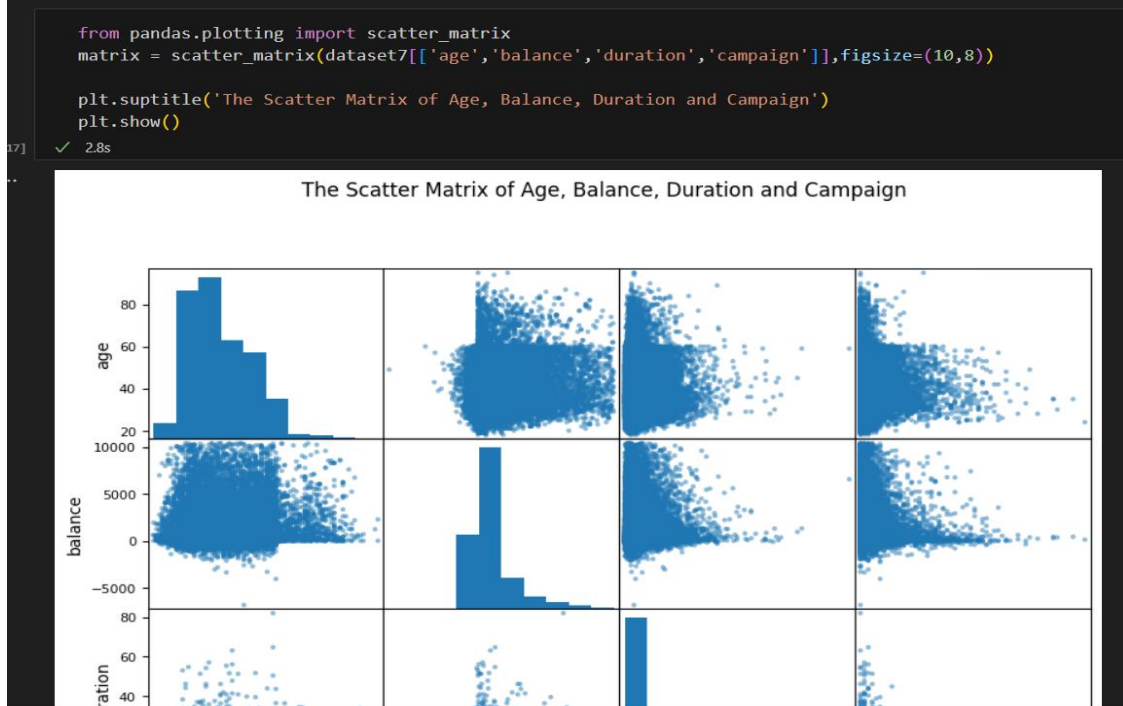


Visualize the distribution of 'duration' & 'campaign'



Scatter matrix and Correlation matrix

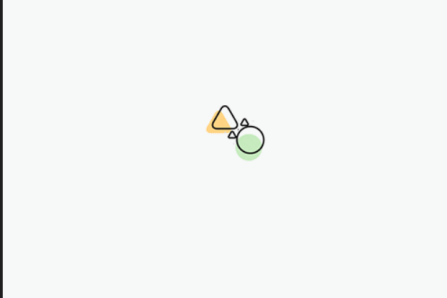
3.5 Scatter matrix and Correlation matrix



Data Visualization

Part 4. Data Visualization

With a sound knowledge of the distribution of key variables, further analysis of each customer characteristic can be carried out to investigate its influence on the subscription rate.



```
Predicting Customer Response to Telemarketing Campaigns.ipynb
D:\> Sem 7 > New > Predicting Customer Response to Telemarketing Campaigns.ipynb > Part 3. Exploratory Data Analysis > 3.5 Scatter matrix and Correlation matrix
+ Code + Markdown | Run All | Restart | Clear All Outputs | Variables | Outline ... Python 3.12.4

count_balance_response_pct = pd.crosstab(dataset7['response'],dataset7['balance_group']).apply(lambda x: x/x.sum() * 100)
count_balance_response_pct = count_balance_response_pct.transpose()

[24] ✓ 0.0s Python

bal = pd.DataFrame(dataset7['balance_group'].value_counts())
bal.rename(columns={'count': 'balance_group'}, inplace=True)
bal['% Contacted'] = bal['balance_group']*100/bal['balance_group'].sum()
bal['% Subscription'] = count_balance_response_pct['yes']
bal.drop('balance_group',axis = 1,inplace = True)

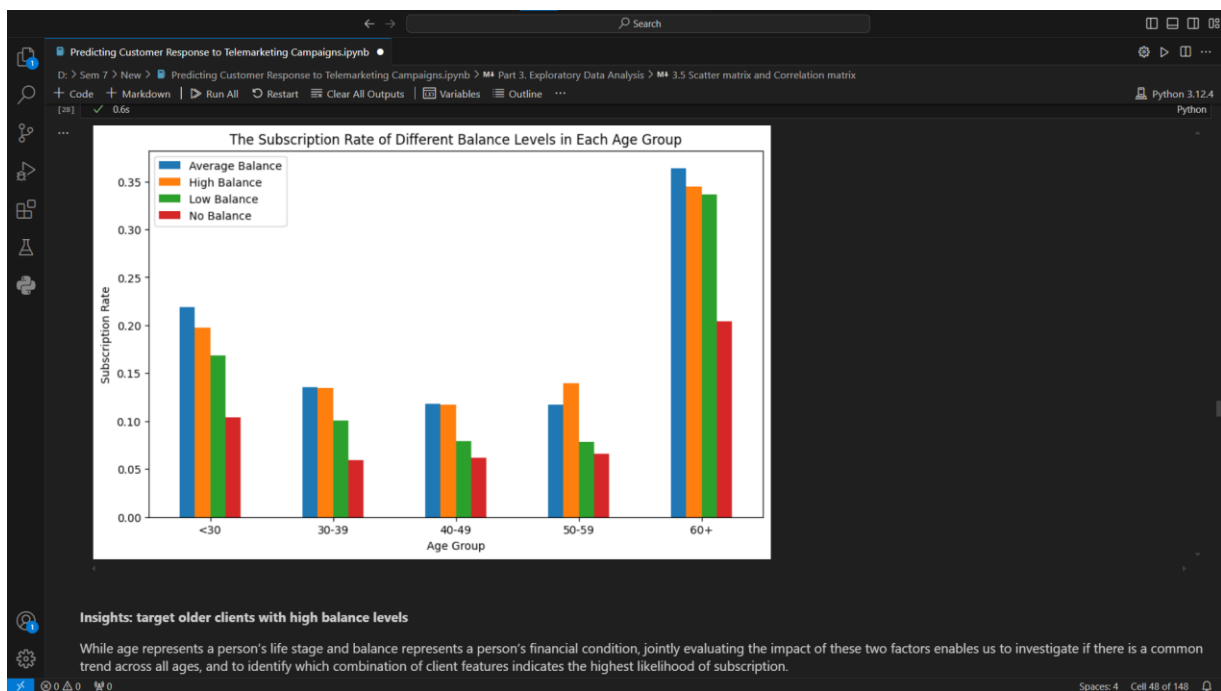
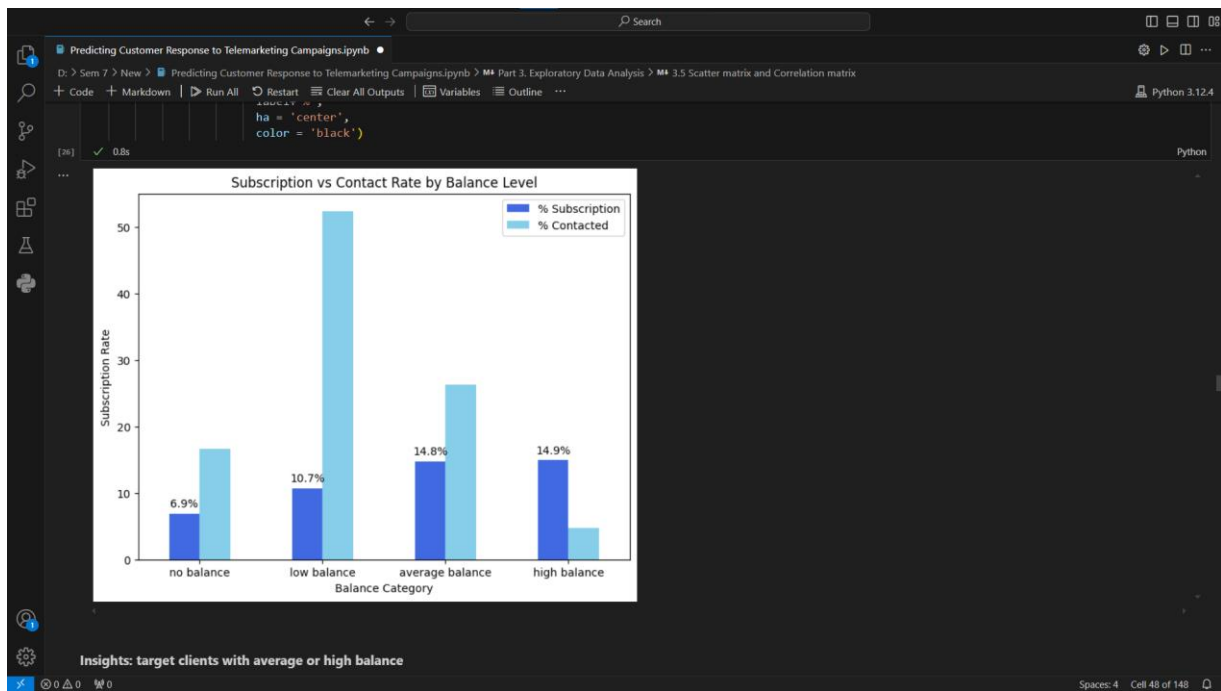
bal['bal'] = [1,2,0,3]
bal = bal.sort_values('bal',ascending = True)

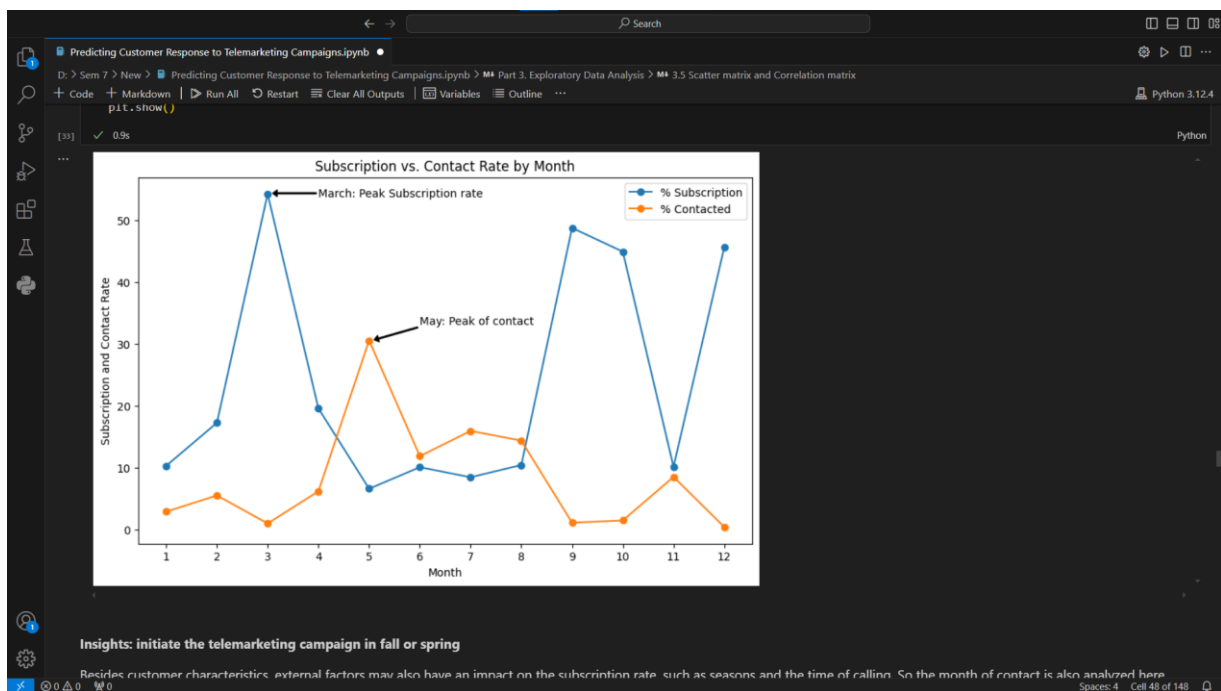
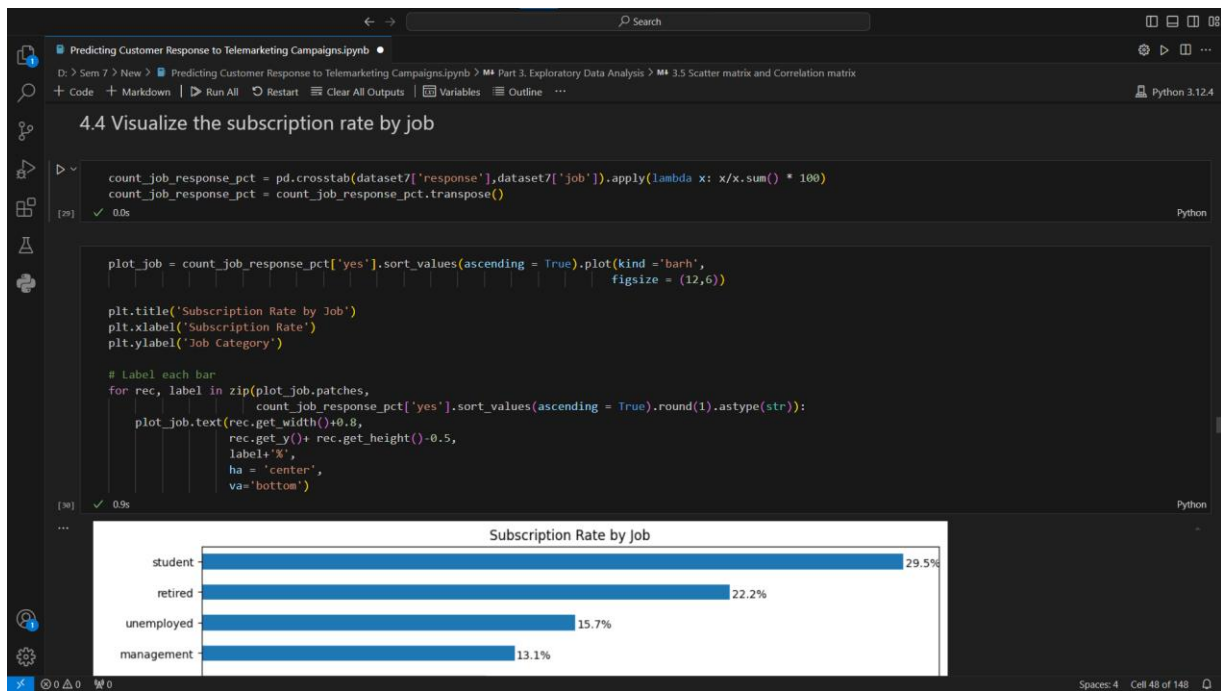
[25] ✓ 0.0s Python

plot_balance = bal[['% Subscription','% Contacted']].plot(kind = 'bar',
                                                         color = ('royalblue','skyblue'),
                                                         figsize = (8,6))

plt.title('Subscription vs Contact Rate by Balance Level')
plt.ylabel('Subscription Rate')
plt.xlabel('Balance Category')
plt.xticks(rotation = 'horizontal')

# label the bar
for rec, label in zip(plot_balance.patches,
                      bal['% Subscription'].round(1).astype(str)):
    plot_balance.text(rec.get_x() + rec.get_width()/2,
                      rec.get_height() + 1,
                      label+'%',
                      ha = 'center',
                      color = 'black')
```





Predicting Customer Response to Telemarketing Campaigns.ipynb

D: > Sem 7 > New > Predicting Customer Response to Telemarketing Campaigns.ipynb > Part 3: Exploratory Data Analysis > 3.5 Scatter matrix and Correlation matrix

+ Code + Markdown | Run All | Restart | Clear All Outputs | Variables | Outline ...

Python 3.12.4

Part 5. Machine Learning: Classification

The main objective of this project is to identify the most responsive customers before the marketing campaign so that the bank will be able to efficiently reach out to them, saving time and marketing resources. To achieve this objective, classification algorithms will be employed. By analyzing customer statistics, a classification model will be built to classify all clients into two groups: "yes" to term deposits and "no" to term deposits.

Load the cleaned dataset

```
linkname = 'bank_cleaned.csv'
dataset = read_csv(linkname)
```

Spaces: 4 Cell 48 of 148

Predicting Customer Response to Telemarketing Campaigns.ipynb

D: > Sem 7 > New > Predicting Customer Response to Telemarketing Campaigns.ipynb > Part 3: Exploratory Data Analysis > 3.5 Scatter matrix and Correlation matrix

+ Code + Markdown | Run All | Restart | Clear All Outputs | Variables | Outline ...

Python 3.12.4

Prepare Data for Classification

5.1 Select variables relevant to customers

Only the most relevant customer information is considered, which includes job title, education, age, balance, default record, housing record and loan record. Other information, such as 'the number of contacts performed before this campaign', is omitted because it is not directly related to customers themselves.

```
dataset.drop(['marital'],axis=1, inplace=True)
dataset1 = dataset.iloc[:, 0:7]
```

[36] ✓ 0.0s Python

5.2 Tranform categorical data into dummy variables

Since machine learning algorithms only take numerical values, all five categorical variables (job, education, default, housing and loan) are transformed into dummy variables.

Dummy variables were used instead of continuous integers because these categorical variables are not ordinal. They simply represent different types rather than levels, so dummy variables are ideal to distinguish the effect of different categories.

```
dataset2 = pd.get_dummies(dataset1, columns = ['job'])
dataset2 = pd.get_dummies(dataset2, columns = ['education'])
dataset2['housing'] = dataset2['housing'].map({'yes': 1, 'no': 0})
dataset2['default'] = dataset2['default'].map({'yes': 1, 'no': 0})
dataset2['loan'] = dataset2['loan'].map({'yes': 1, 'no': 0})
dataset_response = pd.DataFrame(dataset['response_binary'])
dataset2 = pd.merge(dataset2, dataset_response, left_index = True, right_index = True)
```

[37] ✓ 0.0s Python

Spaces: 4 Cell 48 of 148

Predicting Customer Response to Telemarketing Campaigns.ipynb

Compare classification algorithms

Four different classification algorithms (Logistic Regression, K-Neighbors Classifier, Decision Tree Classifier, and Gaussian NB) are run on the dataset and the best-performing one will be used to build the classification model.

```
[41] ✓ 0.5s
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB

models = []
models.append(('LR', LogisticRegression()))
models.append(('KNN', KNeighborsClassifier()))
models.append(('CART', DecisionTreeClassifier()))
models.append(('NB', GaussianNB()))

# Check the data type of your target variable
print(Y_train.dtype)

# If it's not an integer or object (string), you'll need to convert it.
# For example, if your labels are strings, you can use:
Y_train = Y_train.astype(str)

# If you have numerical labels that are not integers, convert them:
Y_train = Y_train.astype(int)

# Now, rerun your cross-validation code:
results_c = []
```

Python

Sorry, something went wrong activating IntelliCode support for Python...

Predicting Customer Response to Telemarketing Campaigns.ipynb

Test LR model on the test set

```
[45] ✓ 0.4s
LR = LogisticRegression()
LR.fit(X_train, Y_train)
```

LogisticRegression

Evaluate LR Model

```
[46] ✓ 0.0s
predictions = LR.predict(X_test)

# Check the data types and unique values in Y_test and predictions
print("Y_test data type:", type(Y_test))
print("Y_test unique values:", np.unique(Y_test))

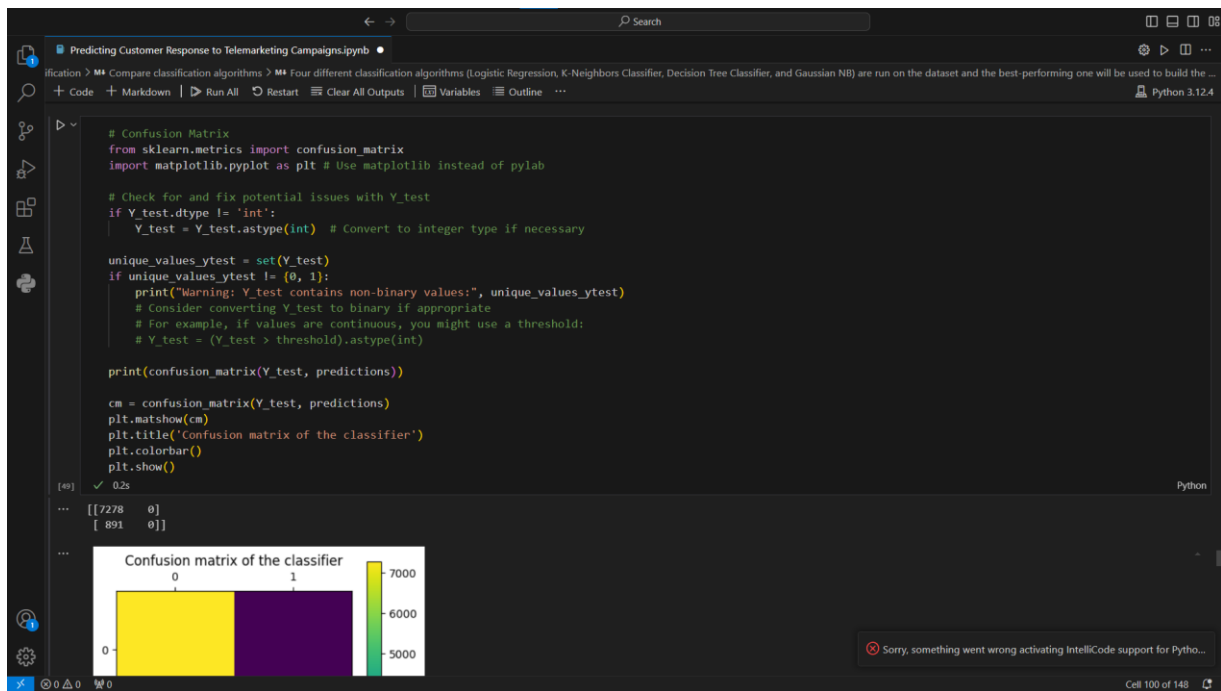
print("\npredictions data type:", type(predictions))
print("predictions unique values:", np.unique(predictions))

# If Y_test is not a 1D array-like object, try to reshape it
if not hasattr(Y_test, 'shape') or len(Y_test.shape) != 1:
    Y_test = Y_test.reshape(-1)

# If necessary, convert Y_test and predictions to a compatible format
# (e.g., using label encoding or one-hot encoding if they are categorical)
```

Python

Sorry, something went wrong activating IntelliCode support for Python...



Predicting Customer Response to Telemarketing Campaigns.ipynb

ification > Compare classification algorithms > Four different classification algorithms (Logistic Regression, K-Neighbors Classifier, Decision Tree Classifier, and Gaussian NB) are run on the dataset and the best-performing one will be used to build the ...

+ Code + Markdown | Run All | Restart | Clear All Outputs | Variables | Outline ... Python 3.12.4

Part 6. Machine Learning: Regression

Regression analysis is carried out to complement the classification result and help the bank better predict campaign outcome based on customer statistics.

Since the duration of a phone call is positively correlated with the campaign outcome, it can serve as another indicator of the possibility of subscription. In this part, regression algorithms will be used to estimate the duration of a phone call, helping the bank better predict subscription rate.

Prepare Data for Regression

```
dataset4 = dataset2.drop(['response_binary'],axis = 1)
dataset4['duration'] = dataset['duration']
```

[51] ✓ 0.0s Python

6.1 Feature selection

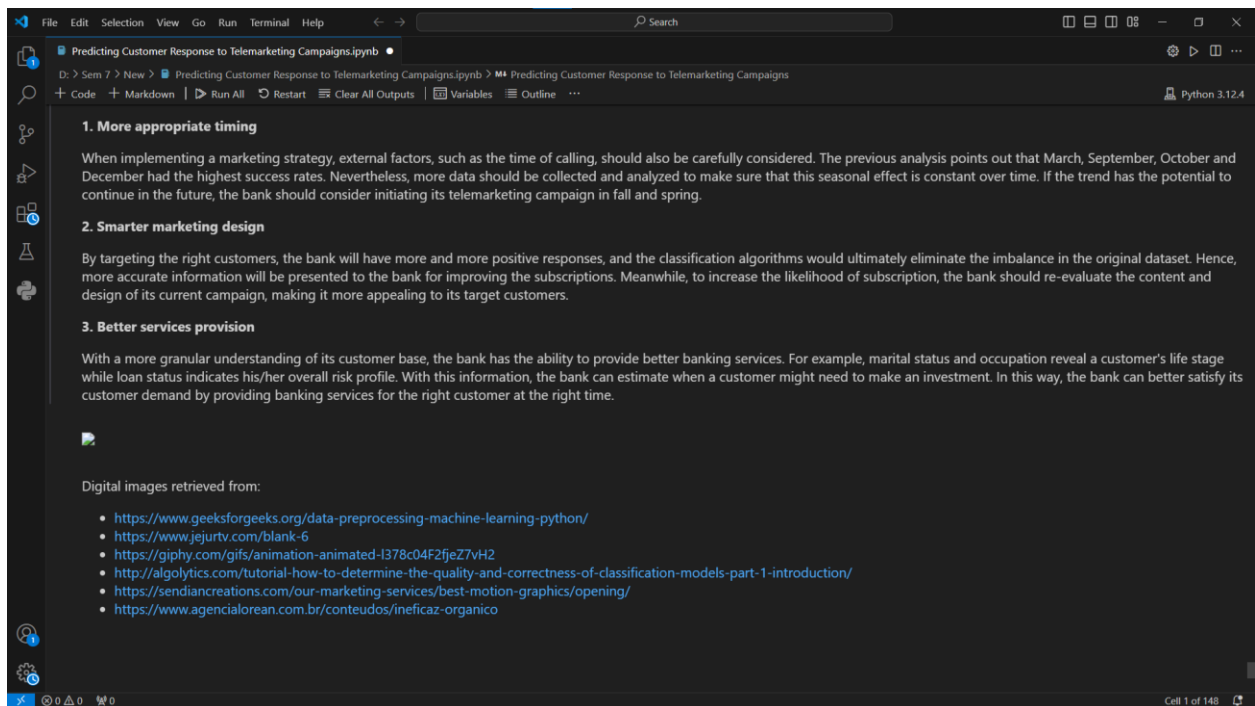
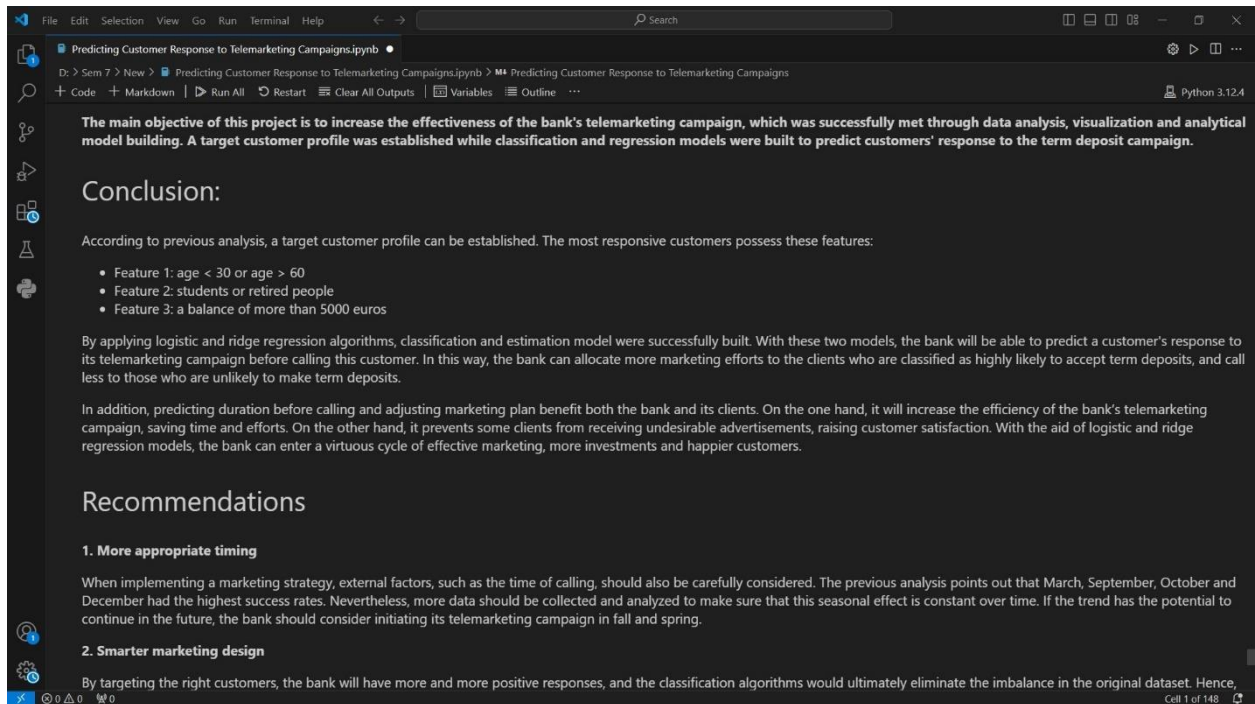
The values of the first 19 columns, which contain customer statistics, are selected as features while the value of the last column, 'duration', is set as target.

```
array = dataset4.values
X = array[:,0:20]
Y = array[:,20]
```

[52] ✓ 0.0s Python

6.2 Train/ test split

Cell 100 of 148



CHAPTER: 4

Conclusion

4.1 Conclusion

Over the course of the 15-day internship, I have gained extensive knowledge and practical experience in data science and machine learning. Each day provided valuable insights and hands-on practice with various tools and techniques essential for this field.

1. Basics of Data Science and Python:

- Understanding the fundamentals of data science and the importance of Python in this domain.
- Learning basic Python programming, including data types and essential libraries like NumPy and Pandas.

2. Data Manipulation and Analysis:

- Practicing data manipulation using Pandas and NumPy, including operations such as generating random numbers, array slicing, concatenation, and sorting.
- Creating and handling DataFrames for comprehensive data exploration and analysis.

3. Data Visualization:

- Utilizing Matplotlib and Seaborn for creating various types of plots, such as line plots, scatter plots, bar plots, histograms, and heatmaps.
- Learning to visualize data effectively for better insights and presentations.

4. Advanced Data Handling:

- Creating pivot tables in Pandas for data summarization and generating detailed statistical summaries.
- Installing and using Pandas Profiling for comprehensive data analysis reports.

5. Machine Learning with Scikit-Learn:

- Understanding regression analysis and implementing regression models using scikit-learn.
- Training and evaluating simple and multiple linear regression models for predictive analysis.

Throughout the internship, I worked on a project titled "Predicting Customer Response to Telemarketing Campaigns," which allowed me to apply the concepts and techniques learned daily. This project provided practical experience in data manipulation, visualization, and building predictive models.

The knowledge and skills acquired during this internship have significantly enhanced my understanding of data science and machine learning. I am now better equipped to tackle real-world data challenges and contribute effectively to data-driven projects. This internship has been a valuable learning experience, laying a strong foundation for my future endeavors in the field of data science.

CHAPTER: 5

Future Enhancement

5.1 Future Enhancement

1. Advanced Machine Learning Techniques:

- Explore more sophisticated machine learning algorithms, such as decision trees, random forests, support vector machines, and neural networks.

2. Hyperparameter Tuning:

- Learn and apply techniques for hyperparameter tuning, such as grid search and random search, to optimize model performance.

3. Model Evaluation and Validation:

- Implement advanced evaluation metrics and cross-validation techniques to ensure the robustness and generalizability of models.

4. Feature Engineering:

- Focus on feature selection and extraction to improve model performance.

5. Time Series Analysis:

- Explore time series data and implement forecasting models if the data involves temporal patterns.

6. Handling Imbalanced Data:

- Learn techniques for handling imbalanced datasets, such as resampling methods and using appropriate evaluation metrics.

These enhancements will further solidify your understanding of data science and machine learning, making you well-prepared for tackling complex data challenges and contributing to impactful projects in the future.